

1.Évaluation des Performances des Systèmes Informatiques : Partie 1

1.1.Résumé Exécutif

L'évaluation de la performance des systèmes informatiques est une discipline fondamentale visant à obtenir des performances supérieures ("plus rapide", "meilleur", "plus fort") au coût le plus bas ("moins cher"). Son importance est cruciale non seulement pour des raisons techniques mais aussi pour son impact direct sur les résultats commerciaux. Des données concrètes démontrent que la latence, même de l'ordre de quelques millisecondes, peut entraîner des pertes de revenus de plusieurs millions de dollars et une baisse significative des taux de conversion des utilisateurs pour des entreprises comme Amazon et Akamai.

Pour caractériser la performance, trois techniques principales sont utilisées : la **Mesure** sur des systèmes réels, la **Simulation** de modèles de systèmes et l'**Analyse Numérique** via des modèles mathématiques. Chacune de ces méthodes présente des avantages et des inconvénients en termes de coût, de précision et de stade d'application. Le principe de la **validation croisée** est essentiel : les résultats d'une technique ne doivent être considérés comme fiables qu'après avoir été validés par au moins une autre méthode.

Une évaluation rigoureuse repose sur une **conception d'expériences** (Experimental Design) méthodique. Cette démarche permet de comprendre comment différents **facteurs** (variables contrôlables) influencent les **métriques** de performance. Si une approche exhaustive comme le **plan factoriel complet** teste toutes les combinaisons possibles, elle est souvent irréalisable en pratique. L'approche "**un facteur à la fois**" est dangereuse car elle peut masquer les interactions critiques entre les facteurs. Une stratégie plus robuste consiste à commencer par un échantillonnage aléatoire pour identifier les facteurs influents avant de mener des analyses plus ciblées.

1.2.L'Impératif de l'Évaluation de la Performance

L'objectif principal de l'évaluation de la performance est de maximiser l'efficacité d'un système tout en minimisant les coûts. Cette démarche répond à plusieurs besoins critiques.

A. Applications Clés

L'évaluation de la performance est nécessaire pour :

- **Améliorer un système existant** : Identifier les goulots d'étranglement et les opportunités d'optimisation.
- **Comparer des conceptions alternatives** : Choisir l'architecture ou l'algorithme le plus performant pour une tâche donnée.
- **Planification de la capacité (Capacity Planning)** : Déterminer les ressources nécessaires pour répondre à une demande future ou à des pics de charge (par exemple, pour le Super Bowl).

- **Respecter les objectifs de niveau de service (Service Level Objective - SLO) :** S'assurer que le système répond aux exigences de performance contractuelles ou attendues.

B. Impact Commercial de la Latence

La performance, et plus particulièrement la latence, a des conséquences financières directes et mesurables.

- **Akamai :** Un retard de 100 millisecondes dans le temps de chargement d'un site web peut réduire les taux de conversion de 7%. De plus, 53% des visiteurs sur mobile quittent une page qui met plus de trois secondes à se charger.
- **Amazon :** Chaque 100 millisecondes de latence supplémentaire leur coûte 1% de ventes.
- **Trading financier :** Un courtier peut perdre 4 millions de dollars de revenus pour chaque milliseconde de retard dans l'exécution d'une transaction.

1.3.Principes Fondamentaux de l'Analyse de Systèmes

Un **système** est défini comme tout processus, machine ou ensemble de matériel et de logiciel qui transforme des entrées en sorties. L'analyse de sa performance nécessite de bien définir ses composants.

A. Terminologie Essentielle

Terme	Définition	Exemple (Serveur Web)
Système	Tout ensemble de matériel, logiciel et firmware.	Le serveur web hébergeant un site.
Facteurs Contrôlables	Variables d'entrée qui peuvent être modifiées intentionnellement.	Nombre de serveurs, logiciel utilisé, paramètres du serveur.
Facteurs Incontrôlables	Variables qui influencent le système mais ne peuvent pas être contrôlées.	Charge du réseau, température de la salle, distance des clients.
Niveaux (Levels)	Les valeurs spécifiques qu'un facteur peut prendre.	CPU de type "68000", "8080", ou "Z80".
Workload (Charge)	L'ensemble des requêtes faites par les utilisateurs du système.	La distribution des utilisateurs, le taux d'arrivée des requêtes.
Variable de Réponse	Le résultat observé de l'expérience, la métrique de performance.	Temps de réponse, débit, taux de conversion.
Expérience (Test)	L'exécution du système pour une combinaison unique de niveaux de facteurs.	Mesurer le temps de réponse avec 1 serveur et un certain logiciel.
Réplication (Run)	Répétition d'une ou plusieurs expériences pour valider les résultats.	Répéter le même test plusieurs fois.

B. Objectifs Spécifiques de l'Analyse

L'évaluation de la performance permet de répondre à des questions précises :

- **Scalabilité** : Combien d'utilisateurs un serveur peut-il gérer ? Comment planifier le nombre de serveurs nécessaires pour une charge croissante ?
- **Fiabilité (Reliability)** : De combien de mémoire un programme a-t-il besoin pour éviter les pannes en ne dépassant pas un certain seuil ?
- **Optimisation** : Quel algorithme est le plus rapide pour une tâche spécifique ?

1.4. Techniques d'Évaluation de la Performance

Il existe trois approches principales pour estimer la performance d'un système. Le choix dépend du stade de développement du projet, du coût et de la précision requise.

A. La Mesure (Measurements)

- **Description** : Observer et collecter des données sur un système réel en fonctionnement. Par exemple, implémenter trois algorithmes de tri et mesurer leur temps d'exécution pour différentes tailles d'entrée.
- **Avantage** : Fournit des résultats réalistes car ils proviennent du système réel.
- **Inconvénient** : Le système doit être construit et opérationnel avant que les mesures puissent être effectuées, ce qui n'est pas possible lors de la phase de conception.

B. La Simulation

- **Description** : Construire un modèle informatique du système et simuler son comportement. Par exemple, créer une chronologie des requêtes et des réponses d'un serveur web.
- **Avantage** : Relativement facile à mettre en œuvre, surtout lorsque la construction du système réel est coûteuse ou impossible.
- **Inconvénient** : Pour les systèmes complexes, la simulation doit s'exécuter pendant une longue période (parfois plusieurs jours) pour produire des résultats fiables.

C. L'Analyse Numérique (ou Analytique)

- **Description** : Développer un modèle mathématique du système pour calculer ses caractéristiques de performance. Par exemple, utiliser la théorie des files d'attente pour calculer le temps de réponse.
- **Avantage** : Rapide et peu coûteuse, ne nécessitant que des compétences analytiques.
- **Inconvénient** : Il est souvent impossible de prendre en compte toutes les composantes et interactions d'un système complexe, ce qui peut limiter la précision.

D. Tableau Comparatif des Techniques

Caractéristique	Analyse Analytique	Simulation	Mesure
Stade d'Application	N'importe lequel	N'importe lequel	Post-implémentation
Temps Requis	Faible	Moyen	Variable
Outils	Analystes	Langages informatiques	Instrumentation
Précision	Faible	Modérée	Élevée (Variable)
Coût	Faible	Moyen	Élevé
Scalabilité	Faible	Moyenne	Élevée

1.5. La Règle d'Or : La Validation Croisée

La fiabilité des résultats est primordiale. Pour cette raison, il est crucial de ne jamais se fier à une seule technique d'évaluation.

- **Règle 1** : Ne faites pas confiance aux résultats d'un modèle analytique tant qu'ils n'ont pas été validés par une simulation ou des mesures.
- **Règle 2** : Ne faites pas confiance aux résultats d'une simulation tant qu'ils n'ont pas été validés par un modèle analytique ou des mesures.
- **Règle 3** : Ne faites pas confiance aux résultats d'une mesure tant qu'ils n'ont pas été validés par une simulation ou un modèle analytique.

En pratique, pour des projets industriels ou étudiants, des mesures accompagnées d'une analyse rigoureuse sont souvent suffisantes. La simulation est fréquemment ajoutée pour valider des mesures à petite échelle et les extrapoler à des systèmes plus grands (par exemple, passer de 8 machines à un datacenter entier).

1.6. Conception d'Expériences (Experimental Design)

La conception d'expériences est le processus structuré qui consiste à planifier des tests pour identifier les raisons des changements observés dans la variable de réponse.

A. Le Plan Factoriel Complet (Full Factorial Design)

Cette approche, également connue sous le nom de "grid search", consiste à tester **toutes les combinaisons possibles** des niveaux de tous les facteurs.

- **Exemple** : Pour un système avec 4 facteurs (CACHE_SIZE avec 4 niveaux, THREADING avec 2 niveaux, ENCRYPTION avec 2 niveaux, SCHEDULER avec 3 niveaux), le nombre total de combinaisons serait de $4 \times 2 \times 2 \times 3 = 48$.
- **Inconvénient** : Le nombre d'expériences devient rapidement ingérable à mesure que le nombre de facteurs et de niveaux augmente.

B. L'Approche "Un Facteur à la Fois" (One-at-a-Time - OAT)

Cette méthode consiste à faire varier un seul facteur à la fois tout en maintenant les autres constants.

- **Risque Majeur** : Cette approche est **très dangereuse** car elle ne permet pas de détecter les **interactions** entre les facteurs. Par exemple, on pourrait conclure à tort que le temps de cuisson du riz a peu d'importance si l'on effectue tous les tests avec une quantité d'eau insuffisante.

C. Stratégie Recommandée et Plans Aléatoires

Une approche plus efficace et robuste se déroule en plusieurs étapes :

1. **Échantillonnage Aléatoire** : Exécuter un nombre limité de tests en choisissant des combinaisons de niveaux de manière aléatoire dans l'espace de conception.

2. **Analyse Initiale** : Utiliser les résultats de cet échantillonnage pour identifier les facteurs qui ont le plus d'impact sur la performance et détecter d'éventuelles interactions.
3. **Expérimentation Ciblée** : Mener une expérience plus approfondie (factorielle ou OAT, cette fois en connaissance de cause) uniquement sur les facteurs jugés importants.

1.7.Étude de Cas : Analyse d'un Serveur Web (File d'attente M/D/1)

Une expérience simple a été menée pour évaluer le temps de service moyen d'un serveur web.

- **Système** : Une file d'attente de type M/D/1 (arrivées Markoviennes, service Déterministe, 1 serveur).
- **Workload (Entrée)** : Taux d'arrivée des requêtes (λ) de 1 requête/seconde, suivant une distribution exponentielle.
- **Capacité du Système** : Taux de service déterministe (μ) de 1.1 requêtes/seconde.
- **Utilisation (ρ)** : $\rho = \lambda/\mu = 1/1.1$.

Les résultats obtenus par les trois techniques montrent une forte convergence, illustrant le principe de validation croisée.

Technique	Temps de Service Moyen (résultat)
Simulation	5.6612
Mesure	5.4608
Analyse Analytique	5.4545

2.Évaluation des Performances des Systèmes Informatique : Partie 2

2.1.Résumé exécutif

L'évaluation de la performance des systèmes est un processus rigoureux qui repose sur une méthodologie structurée englobant la conception expérimentale, la mesure et l'analyse statistique. Le principe fondamental est d'identifier les facteurs qui influencent de manière significative les performances d'un système pour les optimiser. Les métriques clés dans ce domaine sont la latence (temps), le débit (throughput) et l'utilisation des ressources.

La conception expérimentale est une étape critique qui doit impérativement tenir compte des **interactions** entre les facteurs, c'est-à-dire les cas où l'effet d'un facteur dépend du niveau d'un autre. Les approches simplistes, telles que la méthode "un facteur à la fois", sont à proscrire car elles ne parviennent pas à détecter ces interactions et mènent souvent à des conclusions erronées. Des plans plus robustes comme les plans factoriels, aléatoires, et notamment les plans à remplissage d'espace (Maximin), sont préférables pour explorer l'espace des paramètres de manière exhaustive.

L'analyse des résultats s'appuie sur des techniques de **sélection de facteurs** (feature selection) issues de l'apprentissage automatique, telles que la régression linéaire et les arbres de régression. Ces méthodes permettent de quantifier l'influence de chaque facteur et de leurs interactions sur les métriques de performance, aidant ainsi à éliminer les variables non pertinentes et à concentrer les efforts d'optimisation sur ce qui compte vraiment. L'automatisation du flux de travail expérimental, via des outils comme le Network Performance Framework (NPF), est essentielle pour garantir la reproductibilité, la gestion de la complexité et la comparabilité des résultats.

2.2.Le processus de caractérisation de la performance

La caractérisation de la performance suit un processus itératif et structuré qui peut être décomposé en plusieurs étapes interdépendantes.

Étape	Composants	Description
Conception Expérimentale	Paramètres, Facteurs, Charge de travail (Workload), Résumés, Réduction de caractéristiques (PCA)	Définition des variables à tester, de leurs niveaux, et de la manière dont les expériences seront menées pour couvrir l'espace des possibilités.
Méthode de Caractérisation	Mesure, Simulation, Analyse analytique	Choix de la technique pour obtenir les données de performance. La mesure directe est souvent privilégiée.
Résultats	Variables de résultat, Métriques	Collecte des données brutes issues des expériences, centrées sur des indicateurs de performance spécifiques.

Analyse	Sélection de caractéristiques supervisée, Enquête sur les résultats inattendus	Traitement statistique des données pour identifier les facteurs influents, comprendre les relations et les interactions, et valider les hypothèses.
Présentation	Visualisation, Synthèse des données	Communication claire et concise des résultats à travers des graphiques et des résumés pour en tirer des conclusions exploitables.

2.3.Étude de cas illustratif : L'expérience du riz

Une étude de cas sur la cuisson du riz est utilisée pour illustrer les concepts clés de l'évaluation de la performance. L'objectif est de déterminer comment cuisiner un "bon riz".

- **Facteurs Contrôlables** : Quantité d'eau, quantité de sel, temps de cuisson. Un facteur additionnel, la présence d'un couvercle, est également étudié.
- **Métrique de Performance** : Qualité du riz, notée sur une échelle.
- **Observations Clés** :
 - Un manque d'eau (*Low water*) conduit systématiquement à un mauvais résultat.
 - Trop de sel (*Too much salt*) donne un mauvais riz.
 - **Interaction Cruciale** : Une grande quantité d'eau peut "sauver" un riz trop salé, probablement parce que le sel en excès reste dans l'eau qui est ensuite retirée. Cette interaction ne peut être découverte qu'en faisant varier simultanément la quantité d'eau et de sel.
- **Démarche Expérimentale en Plusieurs Phases** :
 1. **Phase d'exploration** : Comprendre l'impact général et l'importance de chaque facteur, ainsi que leurs interactions, pour définir des "conditions normales".
 2. **Phase d'approfondissement** : Une fois les conditions normales et les facteurs clés identifiés, des analyses plus ciblées sont menées pour affiner les réglages optimaux.

2.4.Métriques clés pour les systèmes informatiques

Métriques fondamentales

Trois catégories principales de métriques sont utilisées pour évaluer la performance des systèmes informatiques :

1. **Temps / Latence** : Le temps nécessaire pour accomplir une tâche ou une opération. La latence augmente généralement avec la charge de travail.
2. **Taux / Débit (Throughput)** : La quantité de travail accompli par unité de temps (ex: Gbits/sec). Le débit atteint souvent un plateau lorsque le système sature.
3. **Ressource / Utilisation** : La quantité de ressources (ex: cœurs de CPU, mémoire) consommée pour atteindre une certaine performance. L'objectif est souvent de maximiser la performance pour une quantité de ressources donnée.

Capacité du système

La relation entre la charge et le débit permet de définir plusieurs types de capacités :

- **Capacité nominale** : Le débit dans des conditions idéales.
- **Capacité utilisable** : La capacité maximale avant que le débit n'atteigne un plateau ou ne se dégrade.
- **Capacité du genou (Knee capacity)** : Le point au-delà duquel l'augmentation de la charge n'entraîne plus un gain proportionnel en débit.

Autres métriques pertinentes

- **Fiabilité** : Probabilité d'erreurs.
- **Disponibilité** : Temps moyen avant défaillance (MTTF).
- **Ratio coût/performance**.
- **Efficacité énergétique** : Requêtes par watt.

2.5.Conception expérimentale

Le choix du plan d'expérience est déterminant pour la validité et l'efficacité de l'analyse de performance.

L'importance capitale des interactions

Une **interaction** se produit lorsque l'effet d'un facteur sur la réponse dépend du niveau d'un autre facteur. Par exemple, l'influence du temps de cuisson sur la qualité du riz est quasi nulle s'il n'y a pas d'eau (le riz est toujours brûlé). Ignorer les interactions mène à des conclusions incomplètes ou fausses.

Revue des plans d'expérience

- **Plan "un facteur à la fois" (One-at-a-time)** : Consiste à faire varier un seul facteur à la fois tout en maintenant les autres constants.
 - **Inconvénient majeur** : **Ne permet pas d'étudier les interactions.** L'expérience sur le riz menée avec ce plan serait totalement non concluante.
- **Plan factoriel complet (Full Factorial)** : Teste toutes les combinaisons possibles des niveaux de tous les facteurs.
 - **Avantage** : Permet une analyse complète des effets principaux et des interactions.
 - **Inconvénient** : Le nombre d'expériences devient rapidement ingérable. Un exemple de conception de page web avec 7 facteurs et plusieurs niveaux aboutit à 640 combinaisons, ce qui est irréalisable.
- **Plan aléatoire / Monte Carlo** : Sélectionne un niveau aléatoire pour chaque facteur à chaque exécution.
 - **Avantage** : Utile lorsqu'on a peu de connaissances sur les niveaux pertinents.
 - **Inconvénient** : Peut laisser des zones de l'espace des paramètres inexplorées et peut ré-exécuter des combinaisons identiques.
- **Plans à remplissage d'espace (Maximin)** : Une amélioration du plan aléatoire qui vise à répartir les points de test de manière plus homogène dans l'espace des paramètres. L'heuristique consiste à sélectionner des points qui maximisent la distance euclidienne minimale entre tous les points, assurant ainsi une meilleure couverture.

- **Plans factoriels fractionnaires (2^k)** : Une approche où k facteurs sont étudiés, chacun à seulement deux niveaux (un bas et un haut).
 - **Avantages** : Efficace pour identifier rapidement les facteurs les plus influents en début d'étude.
 - **Limites** : N'est valide que si l'effet des facteurs est **unidirectionnel** (par exemple, plus de cœurs CPU est généralement mieux). Ce plan est inadapté pour des facteurs qui ont un optimum, comme le temps de cuisson du riz (pas assez cuit ou trop cuit sont tous deux mauvais).

2.6. Flux de travail expérimental et outils

Étude de cas : Connexion TCP avec `iperf`

Pour évaluer la performance d'une connexion TCP locale, l'outil `iperf` est utilisé. La première étape consiste à identifier les paramètres pertinents (ex: taille de la fenêtre TCP, nombre de connexions parallèles) en consultant la documentation (`man iperf`).

Gestion du flux de travail

L'expérimentation est un processus itératif. Gérer manuellement les multiples paramètres, l'exécution des tests, la collecte des résultats et la création de graphiques devient rapidement fastidieux et source d'erreurs. Il est donc crucial d'utiliser un gestionnaire de flux de travail pour automatiser ce processus.

Le **Network Performance Framework (NPF)** est un exemple d'outil conçu pour orchestrer et reproduire des expériences réseau. Il permet de définir des variables, leurs plages de valeurs, et les scripts à exécuter, automatisant ainsi le déploiement, l'orchestration et la collecte des données pour une analyse et une visualisation simplifiées.

2.7. Analyse des résultats et sélection des facteurs (Feature Selection)

Objectifs d'une expérience

Une expérience de performance vise plusieurs objectifs :

- Identifier les variables les plus influentes sur la réponse.
- Déterminer les réglages optimaux pour ces variables.
- Trouver les configurations qui minimisent la variabilité de la réponse.
- Identifier les réglages qui rendent le système plus robuste aux facteurs incontrôlables.

Techniques de sélection de facteurs

La sélection de facteurs est utilisée pour éliminer les paramètres qui ont peu ou pas d'influence sur la performance, afin de se concentrer sur les plus importants.

- **Régression Linéaire** : Modélise la sortie comme une combinaison linéaire des facteurs d'entrée. L'équation est de la forme $y = \beta_0 + \beta_1 x_1 + \dots$. La magnitude du

coefficient β_i indique l'importance du facteur x_i . Il est possible d'inclure des termes d'interaction (ex: CPU*PARALLEL) pour en évaluer l'effet.

- **ANOVA (Analysis of Variance)** : Utilisée en complément de la régression, l'ANOVA permet de déterminer si l'influence d'un facteur ou d'une interaction est statistiquement significative (généralement, si la p-value $PR(>F)$ est inférieure à 0.05).
- **Arbres de Régression** : Une alternative non linéaire à la régression. Ils fonctionnent bien avec des effets de seuil et peuvent capturer des relations complexes. L'arbre partitionne les données en fonction des facteurs pour minimiser l'erreur quadratique. L'importance d'un facteur est dérivée de sa contribution à la réduction de l'erreur à travers les divisions de l'arbre.

Avertissement : À ce stade de l'analyse, l'objectif n'est pas de construire un modèle prédictif parfait, mais d'utiliser ces outils pour comprendre le système et identifier les variables méritant une étude plus approfondie. Il ne faut jamais tirer de conclusions finales directement à partir d'un large plan d'expérience, mais plutôt comparer les systèmes dans leurs conditions optimales respectives.

3.Évaluation des Performances des Systèmes Informatiques : Partie 3

3.1.Résumé Exécutif

Ce document synthétise les principes fondamentaux et les méthodologies avancées pour l'évaluation de la performance des systèmes informatiques. L'analyse met en lumière une approche systématique en dix étapes, allant de la définition des objectifs à la présentation des résultats. Un accent particulier est mis sur la caractérisation de la charge de travail (workload), qui est l'entrée déterminante des performances d'un système. Des techniques statistiques univariées (histogrammes, indices de tendance centrale, mesures de variabilité) et multivariées (Analyse en Composantes Principales) sont détaillées pour décrire et simplifier ces charges de travail.

L'importance de la visualisation des données est soulignée comme un outil essentiel pour la synthèse, l'exploration et la communication, tout en mettant en garde contre les mauvaises pratiques qui peuvent induire en erreur. La comparaison rigoureuse des systèmes, notamment via le calcul du "speedup", est également abordée. Enfin, le document identifie les erreurs courantes dans l'évaluation de la performance, des objectifs mal définis à une analyse superficielle, et expose les "crimes du benchmarking" tels que la sélection de données partiales et les jeux de ratios trompeurs. L'adoption de principes rigoureux, comme les critères SMART pour la définition des exigences, est présentée comme indispensable pour une évaluation objective et reproductible.

3.2.Méthodologie Systématique de l'Évaluation de la Performance

Une évaluation de performance rigoureuse repose sur une approche structurée et méthodique. Cette approche permet de s'assurer que les résultats sont pertinents, fiables et reproductibles.

Les Dix Étapes d'une Évaluation Systématique

Le processus complet d'une étude de performance peut être décomposé en dix étapes séquentielles, formant une boucle itérative :

1. **Définir les Objectifs et le Système** : Clarifier le but de l'évaluation et délimiter précisément le système à analyser.
2. **Lister les Services et les Résultats** : Identifier les fonctions principales du système et les issues possibles.
3. **Sélectionner les Métriques** : Choisir des indicateurs quantifiables pertinents pour les objectifs (ex: latence, débit).
4. **Lister les Paramètres** : Énumérer tous les paramètres du système et de la charge de travail qui peuvent influencer la performance.
5. **Sélectionner les Facteurs à Étudier** : Choisir un sous-ensemble des paramètres à faire varier activement durant les expériences.
6. **Sélectionner la Technique d'Évaluation** : Opter pour la mesure, la simulation ou l'analyse analytique.

7. **Sélectionner la Charge de Travail (Workload)** : Définir les entrées qui seront soumises au système.
8. **Concevoir les Expériences** : Planifier les tests à effectuer (ex: plan factoriel complet).
9. **Analyser et Interpréter les Données** : Utiliser des outils statistiques pour extraire des conclusions des données collectées.
10. **Présenter les Résultats** : Communiquer les conclusions de manière claire et non-ambiguë, souvent à l'aide de visualisations.

Les Objectifs d'une Expérience

La définition des objectifs est cruciale et guide l'ensemble du processus. Les buts typiques incluent :

- **Déterminer les variables les plus influentes** : Identifier les facteurs qui ont le plus grand impact sur la performance.
- **Trouver les réglages optimaux** : Déterminer les niveaux de facteurs qui permettent d'atteindre une performance souhaitée.
- **Minimiser la variabilité** : Identifier les configurations qui rendent le système plus stable et prédictible.
- **Améliorer la robustesse** : Trouver des réglages qui minimisent l'impact des facteurs non contrôlables (le "bruit").

Les "3 R" de l'Expérimentation Scientifique

Pour garantir la crédibilité des résultats, trois niveaux de validation sont distingués :

- **Répétabilité (Repeatability)** : La même équipe obtient les mêmes résultats avec le même environnement et le même logiciel.
- **Reproductibilité (Reproducibility)** : Une équipe différente, sur une configuration expérimentale différente, obtient les mêmes résultats en utilisant le logiciel des auteurs originaux.
- **Réplicabilité (Replicability)** : Une équipe différente obtient des résultats similaires (avec une précision définie) sans utiliser le logiciel des auteurs originaux.

3.3.Caractérisation de la Charge de Travail (Workload)

La charge de travail, ou "workload", correspond à l'ensemble des entrées soumises à un système. Sa caractérisation est une étape fondamentale, car la performance dépend majoritairement de ces entrées.

Types et Sélection de Workloads

- **Workload Réel** : Observé sur un système en conditions normales d'opération.
- **Workload Synthétique** : Créé pour imiter un workload réel ou pour tester des cas spécifiques. Il est contrôlable, répétable et ne contient pas de données sensibles.
- **Workload de Test** : Tout workload, réel ou synthétique, utilisé dans une étude de performance.

Lors de la sélection, il faut considérer le **niveau de charge** (pleine capacité, au-delà de la capacité, charge typique) et s'assurer que des **composants externes** ne deviennent pas le goulot d'étranglement, ce qui masquerait les différences de performance du système étudié.

Techniques de Caractérisation

L'objectif est de décrire la charge de travail de manière concise (quelques chiffres, un graphique) pour permettre la génération d'un workload synthétique équivalent.

Statistiques Descriptives Univariées

Pour une seule variable (ex: la taille des paquets dans une trace réseau), on utilise :

- **Visualisation (Histogramme)** : Permet de voir la distribution des données. Le choix du nombre de "bins" (intervalles) est critique ; il est conseillé de commencer avec un grand nombre de bins pour voir la densité réelle avant de simplifier.
- **Indices de Tendance Centrale** :
 - **Moyenne** : La somme des valeurs divisée par leur nombre.
 - **Médiane** : La valeur centrale des données triées (50ème percentile).
 - **Mode** : La valeur la plus fréquente.
 - *Note : Si la distribution est unimodale et symétrique, ces trois indices sont égaux.*
- **Indices de Variabilité** : Doivent toujours accompagner les indices de tendance centrale.
 - **Variance et Écart-Type** : Mesurent la dispersion des données autour de la moyenne.
 - **Quantiles et Intervalle Interquartile (IQR)** : Les quantiles divisent les données triées en parts égales (ex: quartiles). L'IQR (Q75 - Q25) mesure la dispersion des 50% de données centrales.
 - **Visualisation de la Variabilité** : Les **boxplots** sont efficaces pour représenter la distribution et la variabilité, surtout en cas de forte variance.

Modèles de Markov

Lorsqu'un motif séquentiel est observé dans les données (ex: une alternance de paquets longs et courts), un modèle de Markov peut le capturer. Il est défini par une **matrice de transition** qui donne la probabilité de passer d'un état à un autre. Par exemple, pour des paquets classés comme "Longs" (L) ou "Courts" (S) :

From/To	L	S
L	0.701854	0.298146
S	0.698997	0.301003

3.4. Analyse de Données Multivariées : L'Analyse en Composantes Principales (ACP)

Lorsque la charge de travail est décrite par plusieurs variables corrélées (ex: paquets envoyés et paquets reçus), l'Analyse en Composantes Principales (ACP) est une technique puissante de réduction de dimensionnalité non supervisée.

Principe de l'ACP

L'idée clé est de transformer un ensemble de variables possiblement corrélées en un nouvel ensemble de variables non corrélées appelées **composantes principales** (ou facteurs). Ces composantes sont des combinaisons linéaires des variables originales. Elles sont ordonnées de sorte que la première composante explique la plus grande partie de la variance des données.

Étapes de Calcul de l'ACP

Le processus pour trouver ces composantes est le suivant :

1. **Normaliser les Données** : Calculer la moyenne et l'écart-type pour chaque variable et centrer-réduire les données.
2. **Calculer la Matrice de Corrélation (C)** : Cette matrice symétrique contient les coefficients de corrélation de Pearson entre chaque paire de variables. Un coefficient proche de 1 ou -1 indique une forte corrélation linéaire.
3. **Calculer les Valeurs Propres (Eigenvalues)** : Résoudre l'équation $\det(\lambda I - C) = 0$. Les valeurs propres représentent la part de variance expliquée par chaque composante principale.
4. **Calculer les Vecteurs Propres (Eigenvectors)** : Pour chaque valeur propre, trouver le vecteur propre correspondant. Ces vecteurs donnent les "poids" (loadings) de chaque variable originale dans la composition de la nouvelle composante.
5. **Obtenir les Facteurs Principaux** : Multiplier les vecteurs propres par les données normalisées pour obtenir les nouvelles coordonnées des observations dans l'espace des composantes principales.

Dans l'exemple fourni sur les paquets envoyés/reçus, la corrélation est de 0.916. Les valeurs propres sont 1.916 et 0.084. La première composante principale explique 95.8% de la variance, tandis que la seconde n'en explique que 4.3%. On peut donc ignorer la seconde composante et réduire le problème de deux dimensions à une seule, simplifiant ainsi l'analyse.

3.5. Analyse et Présentation des Résultats

Une analyse rigoureuse est inutile si ses résultats ne sont pas communiqués de manière efficace. La visualisation et la synthèse des données sont donc des compétences essentielles.

Visualisation des Données

- **Motivation** : Les graphiques permettent la synthèse de l'information, l'exploration des données, les tests visuels et la communication des résultats.
- **Critères d'un bon graphique** : Il doit être lisible, transmettre un message intelligible et ne laisser place à aucune mauvaise interprétation.
- **Erreurs de visualisation à éviter** :
 - Utiliser des lignes pour des données non continues ou catégorielles (ex: nombre de cœurs CPU).
 - Faire commencer l'axe Y à une valeur autre que 0 sans raison valable.
 - Utiliser des légendes inutiles ou redondantes.
 - Omettre les barres d'erreur ou une indication de la variabilité.

- **Graphiques vs. Statistiques** : Les deux sont nécessaires. Le "Datasaurus dataset" illustre comment des ensembles de données très différents peuvent avoir les mêmes statistiques descriptives, mais des visualisations radicalement différentes.

Synthèse et Comparaison

- **Synthèse** : Utiliser des métriques de tendance centrale et de variabilité est la base. Les **matrices de corrélation** peuvent être visualisées sous forme de cartes de chaleur (heatmaps) pour montrer les relations entre de nombreuses mesures. Les **fonctions de répartition cumulatives (CDF)** sont excellentes pour montrer la distribution complète d'une métrique comme la latence.
- **Comparaison de systèmes** :
 - Présenter les systèmes à comparer sur le même graphique.
 - Utiliser l'axe X pour un facteur important (ex: taille des paquets).
 - Calculer le **Speedup** pour quantifier l'amélioration : $\text{Speedup} = \frac{\text{Temps d'exécution de B}}{\text{Temps d'exécution de A}}$. Un speedup de 1.5 signifie que A est 1.5 fois plus rapide que B.

3.6. Erreurs Courantes et Bonnes Pratiques

L'évaluation de performance est décrite comme un "art" plus qu'une science, car elle est sujette à de nombreuses erreurs et biais.

Erreurs Communes à Éviter

- **Absence d'objectifs clairs** ou objectifs biaisés (ex: "prouver que notre système est meilleur").
- **Métriques de performance incorrectes** ou non pertinentes.
- **Charge de travail non représentative** des cas d'usage réels.
- **Ignorance des interactions** entre les facteurs (privilégier les plans factoriels aux tests "un facteur à la fois").
- **Analyse inexistante ou mauvaise** (un graphique seul ne constitue pas une analyse).
- **Présentation inappropriée** des résultats, rendant l'analyse inutile.
- **Traitement incorrect des valeurs aberrantes** (outliers).
- **Omission des hypothèses et des limitations** de l'étude.

Définir des Exigences SMART

Les exigences de performance vagues comme "le système doit être efficace" sont inutiles. Elles doivent être **SMART** :

- Spécifiques
- Mesurables
- Acceptables
- Réalisables
- Thorough (Exhaustives)

Exemple d'exigence SMART : "Le système doit avoir une latence moyenne inférieure à 50ms (RTT). La latence au 99ème percentile ne doit pas dépasser 200ms."

Les "Crimes du Benchmarking"

Une publication scientifique a identifié 22 "crimes" courants dans les benchmarks de performance. Les plus fréquents sont :

- Ne pas évaluer la dégradation potentielle de la performance.
- Utiliser une baseline de comparaison inappropriée.
- Ne donner aucune indication sur la significativité statistique des données.

Deux exemples notables de manipulation sont :

- **Sélection de données partiales** : Présenter uniquement une plage de données où le système semble bien fonctionner, en cachant les zones où les performances s'effondrent (ex: une base de données qui s'écroule sous une charge élevée).
- **Jeux de Ratios (Ratio Games)** : La comparaison de ratios normalisés par des bases différentes peut mener à des conclusions opposées. Par exemple, normaliser les performances des systèmes A et B par celles de A montrera que B est 1.25 fois plus performant en moyenne, tandis que normaliser par B montrera l'inverse pour A. Les ratios avec des bases différentes ne sont pas comparables.

4. Architecture des ordinateurs, CPU et RAM

4.1. Résumé analytique

Ce document synthétise les principes fondamentaux de l'architecture des ordinateurs, en se concentrant sur l'interaction cruciale entre l'Unité Centrale de Traitement (CPU) et la Mémoire Vive (RAM). Le CPU est présenté comme le cœur du système, responsable de l'exécution des programmes. Sa conception moderne le divise en sous-unités spécialisées, notamment l'Unité Arithmétique et Logique (ALU) pour les calculs et une unité de contrôle pour orchestrer les opérations.

La RAM est une mémoire volatile qui stocke à la fois les instructions du programme (code) et les données nécessaires à son exécution, conformément à l'architecture de Von Neumann. Chaque octet en mémoire est accessible via une adresse unique. Cependant, l'accès à la RAM est une opération intrinsèquement lente par rapport à la vitesse de traitement du CPU. Pour pallier ce goulot d'étranglement, les CPU intègrent des mémoires internes ultra-rapides appelées registres, qui sont utilisées pour stocker temporairement les données et les résultats intermédiaires, minimisant ainsi les accès lents à la mémoire principale.

Le processus de transformation du code source lisible par l'homme en instructions exécutables par la machine est géré par un compilateur. Celui-ci génère un code machine binaire, une séquence d'instructions compactes adaptées à l'Architecture du Jeu d'Instructions (ISA) spécifique du CPU cible. Les ISA varient considérablement, des architectures simples (RISC) aux plus complexes (CISC), ce qui a un impact direct sur la taille du code et l'efficacité de l'exécution. Enfin, les compilateurs emploient diverses techniques d'optimisation (par exemple, utilisation maximale des registres, déroulement de boucles) pour améliorer les performances du code généré, soulignant le lien étroit entre la conception matérielle et la génération de logiciels.

4.2. Analyse détaillée

Les Composants Fondamentaux d'un Ordinateur

Un ordinateur est un système composé de plusieurs éléments interconnectés qui travaillent de concert. Les principaux composants sont :

- **CPU (Central Processing Unit)** : L'unité centrale de traitement, qui exécute les programmes.
- **Mémoire Principale (RAM)** : Une mémoire volatile qui stocke les données et les instructions des programmes en cours d'exécution.
- **Stockage Périphérique Persistant** : Des dispositifs comme les disques durs (HDD) ou les SSD qui conservent les données même sans alimentation électrique.
- **Adaptateur Graphique (avec GPU)** : Une unité spécialisée dans le traitement des graphismes.
- **Autres Périphériques** : Claviers, cartes réseau, et autres dispositifs d'entrée/sortie.

Ces composants communiquent via des bus, notamment un **Bus Mémoire** pour les échanges entre le CPU et la RAM, et un **Bus d'Extension** pour la connexion des autres périphériques.

L'Unité Centrale de Traitement (CPU)

Rôle et Structure

Le CPU est le cerveau de l'ordinateur. Pour simplifier sa conception et améliorer ses performances, il est divisé en plusieurs sous-unités spécialisées :

- **Unités d'exécution (Cores)** : Exécutent les instructions du programme. Les CPU modernes sont souvent multi-cœurs.
- **ALU (Arithmetic Logic Unit)** : Réalise les opérations mathématiques et logiques.
- **Contrôleur Mémoire** : Gère l'accès à la mémoire principale (RAM).
- **Contrôleur d'E/S (Uncore)** : Gère les entrées/sorties et autres communications externes.
- **Gestion des interruptions** : Permet au CPU de réagir aux événements externes.
- **Cache L3 partagé** : Une mémoire cache rapide partagée entre les différents cœurs pour accélérer l'accès aux données fréquemment utilisées.

L'Unité Arithmétique et Logique (ALU)

L'ALU est la sous-unité du CPU qui effectue les calculs. Avant l'intégration moderne, les ALU étaient des puces distinctes. Un exemple historique est la puce 74181, une ALU 4 bits qui prenait deux opérandes de 4 bits (A et B), une entrée de fonction de 4 bits (S) pour sélectionner l'opération à effectuer (ex: $A + B$, $A \wedge B$), et produisait un résultat de 4 bits (F).

La Mémoire Principale (RAM)

Caractéristiques

La RAM (Random-Access Memory) sert à stocker temporairement les données et le code des programmes en cours d'exécution. Ses caractéristiques clés sont :

- **Volatilité** : Les données sont perdues lorsque l'alimentation est coupée.
- **Structure** : Elle est composée de composants électroniques pouvant stocker un état binaire (1 bit). Huit bits sont regroupés pour former un octet (Byte).
- **Accès Aléatoire** : Chaque octet de la mémoire est accessible directement via son adresse, ce qui permet des lectures et écritures rapides à n'importe quel emplacement.

Représentation des Données

La mémoire principale est non typée ; elle ne contient que des suites d'octets. La signification de ces octets est déterminée par le programme.

- **Types de données** : Les types de données qui nécessitent plus d'un octet, comme un `int` de 32 bits en Java, sont stockés sur 4 octets consécutifs. En C, la taille d'un `int` dépend du CPU et du compilateur, mais est souvent de 32 bits.
- **Structures complexes** : Les tableaux et les structures (`struct` en C) sont également stockés sous forme de blocs de mémoire contigus.

Remplissage (Padding)

Pour des raisons de performance, certains CPU accèdent plus rapidement aux données de 32 ou 64 bits si leur adresse est un multiple de 4 ou 8. Pour garantir cet alignement, le compilateur peut insérer des octets de remplissage (padding) inutilisés entre les champs d'une structure de données. Cette pratique est spécifique au CPU et au compilateur.

Exécution d'un Programme

Du Code Source au Code Machine

Un programme écrit dans un langage de haut niveau comme le C est traduit par un compilateur en une séquence d'instructions machine. Ces instructions sont stockées dans la RAM sous un format binaire compact pour des raisons d'efficacité. Par exemple, l'instruction `set32 #3, 1000` est représentée par une séquence d'octets que le CPU peut interpréter.

Fichiers Exécutables et Chargement

Les programmes sont stockés de manière permanente sur un disque dans des fichiers exécutables binaires, dont le format est souvent spécifique au système d'exploitation (par exemple, ELF pour Linux, PE pour Windows). Un fichier exécutable est divisé en plusieurs sections :

Section	Contenu
TEXT	Le code machine binaire du programme.
DATA	Les valeurs initiales des variables globales.
Read-Only	Les données constantes, comme les chaînes de caractères.
BSS	Informations sur l'espace à allouer pour les variables globales nulles.

Pour exécuter un programme, le système d'exploitation le charge depuis le disque vers la RAM. Si l'adresse mémoire préférée par le programme est déjà utilisée, le système effectue une **relocalisation**, c'est-à-dire qu'il modifie les adresses dans le code machine pour qu'elles correspondent au nouvel emplacement en mémoire.

Le Cycle d'Exécution

L'exécution est dirigée par l'unité de contrôle du CPU, qui utilise un registre spécial appelé **Compteur de Programme (PC)** ou **Pointeur d'Instruction (IP)** pour suivre l'adresse de la prochaine instruction à exécuter. Le cycle se déroule comme suit :

1. **Fetch (Lecture)** : L'unité de contrôle lit l'instruction à l'adresse mémoire indiquée par le PC.
2. **Decode (Décodage)** : L'instruction est analysée pour déterminer l'opération à effectuer et ses opérandes.
3. **Lecture des opérandes** : Les données nécessaires sont lues depuis la mémoire ou les registres.
4. **Execute (Exécution)** : L'ALU effectue l'opération.
5. **Write (Écriture)** : Le résultat est écrit en mémoire ou dans un registre.
6. **Mise à jour du PC** : Le PC est avancé à l'adresse de l'instruction suivante.

Ce cycle se répète continuellement.

Sauts (Jumps)

Les structures de contrôle comme les boucles (`while`) et les conditionnelles (`if`) sont implémentées à l'aide d'instructions de saut. Ces instructions modifient la valeur du PC.

- **Saut conditionnel** (`jumpIfGreater`) : Le PC est modifié uniquement si certaines conditions, stockées dans des registres de drapeaux (flags) après une comparaison, sont remplies.
- **Saut inconditionnel** (`jump`) : Le PC est toujours modifié pour pointer vers une nouvelle adresse.

Optimisation des Performances

Le Goulot d'Étranglement de la Mémoire

Il existe une disparité de vitesse massive entre le CPU et la RAM. Une opération de l'ALU peut prendre environ 0,5 nanoseconde, tandis qu'un accès à la RAM peut prendre 100 nanosecondes ou plus. Par conséquent, pour obtenir des programmes rapides, il est impératif de minimiser les accès mémoire.

Les Registres

Pour surmonter la lenteur de la RAM, tous les CPU modernes contiennent des **registres**, qui sont de petites cellules de mémoire extrêmement rapides situées à l'intérieur du CPU.

- **Rôle** : Ils stockent les variables et les résultats intermédiaires pour éviter des allers-retours constants avec la RAM. Un programme utilisant des registres peut réduire considérablement le nombre d'accès mémoire (par exemple, de huit à deux dans un exemple donné).
- **Nombre** : Les CPU modernes possèdent entre 8 et 128 registres. Leur nombre est limité car ils sont très coûteux à fabriquer et un grand nombre de registres augmenterait la taille du code machine.

Architectures de Jeu d'Instructions (ISA)

Définition et Compatibilité

L'**Architecture du Jeu d'Instructions (ISA)** définit l'ensemble des instructions qu'un CPU peut exécuter, le nombre de registres disponibles, et d'autres aspects de son fonctionnement public. Un programme compilé pour une ISA spécifique (par exemple, x86-64 d'Intel) ne peut pas s'exécuter sur un CPU avec une autre ISA (par exemple, ARMv7). C'est le principe de la **compatibilité binaire**.

Modes d'Adressage

Les CPU traitent les adresses comme des nombres, ce qui permet de manipuler directement des pointeurs. Pour accéder efficacement à des structures de données complexes, les CPU supportent divers **modes d'adressage** :

Mode d'adressage	Description	Exemple (ARM)
Absolute	L'adresse est une constante dans l'instruction.	LDR R0, MEM
Literal	L'opérande est une valeur immédiate.	MOV R0, #15
Indexed, base	L'adresse est contenue dans un registre (adressage indirect).	LDR R0, [R1]
Pre-indexed	L'adresse est calculée à partir d'un registre plus un décalage.	LDR R0, [R1, #4]
Double Reg indirect	L'adresse est calculée à partir de la somme de deux registres.	LDR R0, [R1, R2]

Ces modes spécialisés permettent de traduire efficacement des opérations comme l'accès à un champ de structure (`myBankAccount->value`) ou à un élément de tableau (`a[i]`).

RISC contre CISC

Il existe deux philosophies principales dans la conception des ISA :

- **RISC (Reduced Instruction Set Computer) :**
 - **Philosophie :** Utilise un jeu d'instructions simple et réduit.
 - **Caractéristiques :** Seules quelques instructions (`load`, `store`) accèdent à la mémoire. Toutes les autres opérations (calculs) se font uniquement entre registres.
 - **Avantage :** L'unité de contrôle est plus simple (circuit câblé).
- **CISC (Complex Instruction Set Computer) :**
 - **Philosophie :** Propose des instructions complexes et puissantes pour réduire la taille des programmes.
 - **Caractéristiques :** Les instructions peuvent effectuer des opérations complexes qui combinent calculs et accès mémoire (ex: `add r0, [8+r1*4]`).
 - **Implémentation :** Ces instructions complexes sont souvent décomposées en une série de micro-opérations internes (microcode).

Aujourd'hui, de nombreux CPU modernes sont des hybrides, combinant des instructions câblées et micro-codées.

Instructions à 2 ou 3 Opérandes

- **2 opérandes (typique d'Intel) :** L'instruction a deux opérandes, où l'un sert à la fois de source et de destination. Ex: `mult32 #2, r2` (calcule $r2 = r2 * 2$).
- **3 opérandes (typique d'ARM) :** L'instruction spécifie deux sources et une destination distincte. Ex: `mult32 #2, r0, r2` (calcule $r2 = r0 * 2$). Les programmes peuvent nécessiter moins d'instructions, mais chaque instruction est plus grande.

Optimisations du Compilateur

Le compilateur joue un rôle essentiel dans la performance finale d'un programme. Il peut traduire le même code source de nombreuses manières différentes en utilisant des techniques d'optimisation. Les compilateurs comme `gcc` proposent plusieurs niveaux d'optimisation (par exemple, `-O0`, `-O1`, `-O2`, `-O3`).

Parmi les optimisations courantes, on trouve :

- **Allocation de registres** : Conserver les variables dans les registres plutôt qu'en mémoire.
- **Déroulement de boucle et inlining de fonction** : Réduire le surcoût des sauts et des appels de fonction en dupliquant le code.
- **Ajout de padding** : Aligner les données pour des accès plus rapides.
- **Réorganisation des instructions** : Éviter les dépendances qui pourraient ralentir le pipeline d'exécution du CPU.

Le choix des optimisations est un compromis. Certaines augmentent la taille du fichier binaire, d'autres dépendent du CPU cible, et les optimisations les plus "agressives" peuvent parfois modifier le comportement du programme, notamment dans des contextes multi-threadés.

5.Principes de la Simulation de Systèmes

5.1.Résumé Analytique

Ce document synthétise les concepts fondamentaux, les techniques et les outils liés à la simulation de systèmes informatiques, en se basant sur une analyse de la performance. La simulation est présentée comme un outil essentiel pour l'expérimentation, notamment lorsqu'il est impossible de maîtriser l'ensemble des paramètres d'un système complexe. Les simulations sont classées selon deux axes : le temps (continu ou discret) et l'état (continu ou discret), chaque combinaison étant adaptée à des problèmes spécifiques comme la logique numérique, les systèmes de files d'attente ou les circuits analogiques.

Un point central du document est la distinction critique entre la **vérification** et la **validation** d'un modèle. La vérification est assimilée au débogage du simulateur lui-même, tandis que la validation consiste à s'assurer que le modèle correspond fidèlement au comportement du monde réel. Plusieurs techniques de vérification sont détaillées, incluant la conception modulaire, l'intégration de mécanismes d'auto-vérification ("anti-bugging"), les revues de code, l'utilisation de traces d'événements et d'affichages graphiques.

La distinction entre **simulation** et **émulation** est également soulignée. La simulation vise à reproduire le *comportement* d'un système pour en analyser la performance, souvent sans contrainte de temps réel. L'émulation, en revanche, a pour but de reproduire la *fonctionnalité* d'un système, en prenant des raccourcis sur les détails internes pour permettre une utilisation interactive, comme dans le cas des émulateurs de consoles de jeux. Enfin, le document fait référence à divers frameworks de simulation, qu'ils soient génériques (SimPy) ou spécifiques à un domaine (NS-3 pour les réseaux, gem5 pour les CPU).

5.2.Introduction à la Simulation

La simulation est une approche fondamentale dans l'analyse de la performance des systèmes informatiques. Elle s'inscrit dans un processus d'expérimentation qui comporte au moins deux phases. L'un des principaux avantages de la simulation est qu'elle permet d'étudier des systèmes complexes pour lesquels il est impossible d'avoir une compréhension exhaustive de tous les paramètres possibles.

Les principaux sujets abordés incluent :

- La simulation de CPU.
- Les stratégies de mise en cache (FIFO, LRU, LFU).
- Les caches CPU, leur associativité et les problèmes de contention.

5.3.Typologies de Simulation

Les modèles de simulation peuvent être classés selon deux dimensions principales : le temps et l'état.

- **Modèle à temps continu (Continuous Time Model)** : L'état du système est défini à tout instant.

- **Modèle à temps discret (Discrete Time Model)** : L'état du système n'évolue qu'à des instants précis (par ex., à chaque tour de jeu ou coup d'horloge).
- **Modèle à état discret (Discrete State)** : Les variables d'état ne peuvent prendre qu'un nombre fini ou dénombrable de valeurs. Dans le pire des cas, l'état complet de la mémoire d'un ordinateur est une variable discrète.
- **Modèle à état continu (Continuous State)** : Les variables d'état peuvent prendre n'importe quelle valeur dans un intervalle continu.

Le tableau suivant illustre les combinaisons de ces modèles avec des exemples concrets :

	État Discret	État Continu
Temps Discret	• Logique numérique (états binaires, horloge) • Jeux de société (état = position, temps = tours)	Simulations numériques de systèmes analogiques
Temps Continu	• Systèmes de files d'attente • Simulation informatique	• Circuit RC, pendule → Équations différentielles

5.4.Exemple Pratique : Simulation d'un CPU

La simulation d'un processeur (CPU) constitue un exemple concret et fondamental. Étant donné une mémoire et l'emplacement d'un programme, il est possible de construire un simulateur qui exécute ce programme en suivant une boucle d'instructions.

Cycle d'exécution de base :

1. **Initialisation** : Le compteur de programme (PC) est initialisé à l'adresse de départ (ex: PC = 2000).
2. **Boucle d'exécution** :
 - **Fetch (Recherche)** : Lire l'instruction à l'adresse pointée par le PC.
 - **Decode (Décodage)** : Analyser l'opcode de l'instruction.
 - **Execute (Exécution)** : Lire les opérandes, exécuter l'opération via l'ALU, écrire les résultats.
 - **Advance (Avancement)** : Incrémenter le PC de la taille de l'instruction (PC += instruction_size).

Une extension de cet exercice consiste à prendre en compte le **temps**. Par exemple, un cycle d'horloge peut durer 0,5 ns, tandis qu'une lecture en mémoire peut prendre 200 ns, introduisant des délais réalistes dans le modèle.

5.5.Vérification et Validation des Modèles

La création d'un modèle de simulation fiable repose sur deux processus distincts mais complémentaires : la vérification et la validation.

- **Vérification** : Concerne le débogage du simulateur. Il s'agit de s'assurer que le programme de simulation est correctement implémenté.
- **Validation** : Concerne la correspondance entre le modèle et le monde réel. Il s'agit de s'assurer que le simulateur représente fidèlement le système qu'il est censé modéliser.

Il existe quatre scénarios possibles pour un modèle :

1. **Non vérifié, Invalide** : Le simulateur contient des bugs et ses résultats ne correspondent pas aux mesures réelles.
2. **Non vérifié, Valide** : Le simulateur produit des résultats corrects par hasard, mais il contient des bugs non découverts.
3. **Vérifié, Invalide** : Le code du simulateur est correct, mais le modèle sous-jacent ne représente pas la réalité.
4. **Vérifié, Valide** : Le code est correct et le modèle représente fidèlement le système réel.

Techniques de Vérification

Plusieurs techniques peuvent être employées pour vérifier un modèle de simulation :

1. **Conception modulaire descendante (Top Down Modular Design)** : Appliquer le principe "diviser pour régner". Le système est décomposé en modules (sous-programmes, procédures) avec des interfaces bien définies. Chaque module peut être développé, débogué et maintenu indépendamment. Pour un CPU, cela consiste à tester séparément la mémoire, l'ALU, etc.
2. **Techniques anti-bugs (Anti-bugging)** : Intégrer des auto-vérifications dans le code. Par exemple, vérifier que la somme des probabilités est égale à 1, ou que le nombre de tâches restantes est égal au nombre de tâches générées moins celles servies. Dans le cas d'un CPU, cela peut prendre la forme de tests unitaires (`Memory.write(A)` suivi de `Memory.read()` doit retourner A).
3. **Revue de code structurée (Structured Walk-Through)** : Expliquer le code à une autre personne ou à un groupe. Ce processus force à clarifier sa pensée et permet de détecter des erreurs, même si l'interlocuteur est passif.
4. **Exécution de cas simplifiés (Run Simplified Cases)** : Tester le modèle avec des scénarios très simples pour lesquels le résultat est connu ou facile à prévoir (par ex., un seul paquet, une seule source de trafic).
5. **Trace** : Afficher une séquence chronologique des événements importants durant la simulation (ex: 1,1s : Job 7 enters queue). Pour un CPU, une trace peut montrer l'évolution du PC et de la mémoire à chaque cycle.
6. **Affichages graphiques en temps réel (On-Line Graphic Displays)** : Visualiser l'évolution de la simulation. Cela rend la simulation plus intéressante, aide à communiquer ("vendre") les résultats et offre une vue plus complète qu'une simple trace textuelle.
7. **Indépendance par rapport à la graine aléatoire (Seed Independence)** : S'assurer que la simulation produit des résultats statistiquement similaires pour différentes graines (valeurs initiales) du générateur de nombres aléatoires.

Validation du Modèle

La validation consiste à comparer les résultats du simulateur à des résultats connus et mesurés dans le monde réel pour les mêmes entrées (charges de travail ou "workloads").

- **Processus de validation** :
 1. Construire le modèle (simulateur) en se basant sur des observations réelles (par ex., `wll_real`).

2. Comparer les sorties du simulateur (`w11_simulated`) avec les données réelles. C'est l'équivalent d'un "training set".
 3. Tester le modèle avec des charges de travail différentes (`w12`) pour lesquelles on dispose aussi de données réelles (`w12_real`). C'est l'équivalent d'un "validation set".
- **Mesure de la correction** : Des métriques statistiques comme le score R^2 (`r2_score` de `sklearn.metrics`) peuvent être utilisées pour quantifier la correspondance entre les données simulées et réelles.
 - **Validation croisée (Cross-validation)** : Doit être utilisée chaque fois que possible pour renforcer la robustesse du modèle.

Pour l'exemple du CPU, la validation consisterait à vérifier si l'exécution simulée du Programme A produit bien la même sortie que son exécution réelle, et de répéter le processus pour un Programme B.

5.6. Outils et Frameworks de Simulation

Il existe de nombreux outils pour la simulation, allant des frameworks génériques aux simulateurs spécialisés.

- **Frameworks génériques** :
 - **SimPy** : Une bibliothèque Python pour la simulation d'événements discrets.
 - **Simulink (MathWorks)** : Un environnement de programmation graphique pour la modélisation, la simulation et l'analyse de systèmes dynamiques.
 - **Arena** : Un logiciel de simulation d'événements discrets, souvent utilisé pour la modélisation de processus industriels et logistiques.
- **Simulateurs spécifiques à un domaine** :
 - **NS-3** : Un simulateur d'événements discrets largement utilisé pour la recherche sur les réseaux informatiques.
 - **gem5** : Un framework de simulation modulaire pour la recherche en architecture informatique et systèmes d'exploitation.

5.7. Simulation vs. Émulation

Il est crucial de ne pas confondre simulation et émulation.

Simulation

- **Objectif** : Reproduire le **comportement** d'un système.
- **Finalité** : Étudier et obtenir une approximation correcte de la **performance** du système (un "jumeau numérique").
- **Contrainte de temps** : Généralement, ne vise pas à s'exécuter "en temps réel". Une simulation de la dynamique des fluides peut tourner pendant des heures pour modéliser quelques secondes de réalité.

Émulation

- **Objectif** : Reproduire la **fonctionnalité** d'un système.
- **Finalité** : Créer un système équivalent qui soit **fonctionnel** pour l'utilisateur.

- **Contrainte de temps** : Vise souvent une exécution en temps réel ou quasi-réel.
- **Détails internes** : Prend des raccourcis et n'a pas besoin de reproduire fidèlement les composants internes (ex: caches CPU, latence mémoire). L'exemple typique est un émulateur de PlayStation 2, où l'objectif est simplement de pouvoir jouer aux jeux.

6. Mise en Cache (Caching)

6.1. Résumé Exécutif

La mise en cache est une technique d'optimisation fondamentale en informatique, conçue pour combler l'écart de performance significatif entre la vitesse de traitement des processeurs (CPU) et la latence d'accès à la mémoire principale. Elle repose sur l'utilisation d'une mémoire intermédiaire, plus petite et plus rapide (le cache), pour stocker des copies des données les plus fréquemment utilisées provenant d'un espace de stockage plus grand et plus lent (le magasin de sauvegarde).

L'efficacité de ce mécanisme découle du principe de **localité des références**, qui observe que les programmes ont tendance à accéder de manière répétée aux mêmes données (localité temporelle) ou à des données adjacentes (localité spatiale) sur de courtes périodes. La performance d'un cache est mesurée par son **taux de succès** ("hit rate"), soit la proportion de requêtes de données satisfaites directement par le cache.

La conception d'un cache implique un compromis entre le coût, la taille et la vitesse. Pour gérer le contenu de cet espace limité, divers algorithmes de remplacement sont utilisés lorsque le cache est plein. Les principales stratégies incluent :

- **OPTIMAL** : L'approche théorique parfaite, mais irréalisable, qui retire l'élément qui sera utilisé le plus tardivement.
- **FIFO (First-In, First-Out)** : Simple à implémenter mais peut souffrir de l'anomalie de Bélády, où l'augmentation de la taille du cache peut dégrader les performances.
- **LRU (Least Recently Used)** : Retire l'élément le moins récemment utilisé, exploitant la localité temporelle. C'est un algorithme "stack" dont les performances s'améliorent avec la taille.
- **LFU (Least Frequently Used)** : Retire l'élément le moins fréquemment utilisé, mais est sensible à la "pollution du cache".

En pratique, le choix de l'algorithme dépend du contexte : les systèmes de haut niveau (serveurs web, bases de données) privilégient des algorithmes complexes comme LRU pour minimiser le coût élevé d'un échec ("miss"), tandis que les caches de bas niveau (CPU) utilisent des algorithmes plus rapides et simples, comme Random ou des approximations de LRU, pour maintenir une surcharge minimale.

6.2. Le Problème Fondamental et la Solution de la Mise en Cache

Le Goulot d'Étranglement de l'Accès Mémoire

Le cœur du problème de performance dans les systèmes informatiques modernes réside dans la disparité de vitesse entre le CPU et la mémoire principale. Un accès à un registre du CPU s'effectue en une fraction de nanoseconde, tandis qu'un accès à la mémoire principale peut prendre des centaines de fois plus de temps.

- **Accès à un registre (CPU à 2 GHz) : 0,5 ns**
- **Accès à la mémoire principale : 100 ns**

Bien que les registres permettent de réduire le nombre d'accès mémoire, leur capacité est très limitée et insuffisante pour stocker de grands ensembles de données, comme une base de données d'employés d'une grande entreprise. Ils ne peuvent pas anticiper quelles données spécifiques seront demandées.

Définition et Principe du Cache

Un cache est un composant mémoire qui stocke des données afin que les accès futurs à ces mêmes données puissent être exécutés avec un coût réduit, principalement en termes de temps d'accès (latence). Il agit comme un intermédiaire entre une unité de traitement rapide (comme le CPU) et un stockage de données plus lent (comme la mémoire principale).

- **Définition :** "Un composant mémoire qui stocke des données, afin que les accès futurs à ces données puissent être exécutés à moindre coût."

Le cache utilise des puces mémoire plus rapides (SRAM) que la mémoire principale (DRAM), mais celles-ci sont également beaucoup plus chères et donc utilisées en plus petite quantité. L'objectif est de conserver une copie des données "les plus fréquemment" utilisées de la mémoire principale pour réduire le temps d'accès global.

6.3.La Mise en Cache comme Technique d'Optimisation Générale

Le concept de mise en cache n'est pas limité à l'interaction CPU-mémoire ; c'est une technique d'optimisation universelle appliquée à de nombreux niveaux d'un système informatique.

Exemples d'Applications

Cache Disque (Disk Cache)

Le système d'exploitation utilise une partie de la mémoire principale (DRAM) comme cache pour les données provenant de dispositifs de stockage permanents plus lents (disques durs ou SSD). Il conserve une copie des parties de fichiers les plus fréquemment consultées.

Caractéristique	Cache Fichier (DRAM)	Stockage Permanent (HDD / M.2 SSD)
Usage	Cache de fichiers	Stockage permanent
Latence	50 – 100 ns	1 – 10 ms (HDD) / 100 µs (SSD)
Débit	0,5 – 12,8 GB/s	100 – 550 MB/s (HDD) / 3 GB/s (SSD)
Taille typique	≤ 256 GB	≥ 1 TB
Prix/GB	5 €	0,05 €

Cache de Navigateur Web (Web Browser Cache)

Le navigateur web conserve une copie locale des pages web et de leurs composants (images, scripts) fréquemment visités. Cela permet de réduire plusieurs coûts :

- **Temps d'accès** : L'accès local est quasi-instantané comparé à une requête réseau. (Exemple : < 1 ms en local contre 200 ms vers un serveur aux USA).
- **Charge du serveur** : Le serveur distant n'a pas à traiter la requête.
- **Trafic réseau** : Réduit la quantité de données transférées sur Internet.

6.4.Principes Fondamentaux de l'Efficacité du Cache

L'efficacité d'un cache repose sur des comportements prévisibles dans les schémas d'accès aux données.

Localité des Références (Locality of References)

Les programmes ne sollicitent pas les données de manière aléatoire. Leur comportement d'accès, représenté par une **chaîne de références** (une séquence d'accès à des objets de données), suit deux principes clés :

- **Localité Temporelle** : Un programme qui accède à un objet X à un instant t a de fortes chances d'y accéder à nouveau dans un futur proche.
- **Localité Spatiale** : Un programme qui accède à un objet X à un instant t a également de fortes chances d'accéder à un objet "voisin" (X+1) dans un futur proche.

Ce comportement est typique dans presque tous les systèmes informatiques, que ce soit pour l'accès à des variables, des fichiers, des lignes de base de données ou des pages web.

Mesure de l'Efficacité : Taux de Succès et d'Échec

L'efficacité d'un cache se mesure par sa capacité à fournir les données demandées.

- **Succès (Hit)** : Les données demandées se trouvaient dans le cache.
- **Échec (Miss)** : Les données demandées n'étaient pas dans le cache et ont dû être récupérées depuis le magasin de sauvegarde.

Le **taux de succès (Hit Rate)** est la métrique principale : $\text{Taux de Succès} = \frac{\text{Nombre de Succès}}{\text{Nombre total d'accès}}$

Un programme est considéré "**cache-friendly**" s'il opère sur un petit "**working set**" (ensemble de données actives), ce qui permet d'atteindre un taux de succès élevé même avec un cache de taille modeste. À l'inverse, un programme avec un grand "working set" est "**cache-unfriendly**".

Calcul du Temps d'Accès Moyen

Le temps d'accès moyen ($\bar{a}$) est une moyenne pondérée des temps d'accès en cas de succès et d'échec :

$$\bar{a} = r_{\text{hit}} \cdot a_{\text{hit}} + (1 - r_{\text{hit}}) \cdot a_{\text{miss}}$$

Où :

- `r_hit` est le taux de succès.
- `a_hit` est le temps pour accéder aux données depuis le cache.
- `a_miss` est le temps pour accéder aux données depuis le magasin de sauvegarde et les placer dans le cache (pénalité d'échec).

6.5.Algorithmes de Gestion de Cache

Étant donné que les caches sont plus petits que les magasins de sauvegarde, une gestion intelligente de leur contenu est cruciale. L'analyse se concentre sur les caches auto-gérés, où le système décide quels objets conserver.

Stratégies de Remplacement (Removal Policies)

Lorsque le cache est plein et qu'un nouvel objet doit être ajouté, un objet existant doit être expulsé. Le choix de cet objet est déterminé par la politique de remplacement.

OPTIMAL (Algorithme Optimal)

- **Principe** : Retire l'objet qui ne sera pas utilisé pendant la plus longue période dans le futur.
- **Analyse** : C'est la meilleure stratégie possible, servant de référence théorique pour évaluer les autres algorithmes. Elle est cependant **impossible à mettre en œuvre en pratique**, car elle nécessite de connaître le futur.

FIFO (First-In, First-Out)

- **Principe** : Retire l'objet le plus ancien (le premier à être entré).
- **Avantages** : Très simple et peu coûteux à implémenter (via une file d'attente ou un tampon circulaire).
- **Inconvénients** : Souffre de l'**Anomalie de Bélády**, un phénomène contre-intuitif où l'augmentation de la taille du cache peut entraîner une diminution du taux de succès. Pour cette raison, FIFO n'est pas un **algorithme "stack"**.

LRU (Least Recently Used)

- **Principe** : Retire l'objet qui n'a pas été utilisé depuis le plus longtemps. Cette stratégie repose sur l'hypothèse de la localité temporelle.
- **Avantages** : Offre de bonnes performances et est un **algorithme "stack"**, ce qui garantit que le taux de succès s'améliore (ou reste stable) avec l'augmentation de la taille du cache.
- **Inconvénients** : Son implémentation peut être coûteuse pour les caches de bas niveau (CPU), car elle nécessite de mettre à jour des structures de données à chaque accès.

LFU (Least Frequently Used)

- **Principe** : Retire l'objet ayant le plus faible compteur d'accès. Chaque objet dans le cache a un compteur qui est incrémenté à chaque accès.
- **Avantages** : Est également un algorithme "stack".

- **Inconvénients** : Génère plus de surcharge que LRU (gestion des compteurs) et est vulnérable à la "**pollution du cache**", où des objets accédés très fréquemment dans le passé mais désormais inutiles restent indéfiniment dans le cache, bloquant la place pour de nouvelles données pertinentes.

LFU avec Vieillessement (LFU with Aging)

- **Principe** : Une variation de LFU pour contrer la pollution du cache. Périodiquement, les compteurs de tous les objets du cache sont réduits (par exemple, divisés par deux).
- **Analyse** : Ce mécanisme de "vieillessement" permet aux objets inutilisés, même ceux avec un compteur élevé, de perdre progressivement de leur importance et d'être finalement expulsés. L'inconvénient est une complexité accrue et la nécessité de régler un paramètre de seuil.

6.6.Algorithmes "Stack" et Applications Pratiques

Le Concept d'Algorithme "Stack"

Les algorithmes "stack" (comme OPTIMAL, LRU, et LFU) possèdent une propriété d'inclusion essentielle : l'ensemble des éléments présents dans un cache de taille N est toujours un sous-ensemble des éléments présents dans un cache de taille $N+1$ pour la même chaîne de références.

- Propriété : $E_S(N) \subseteq E_S(N + 1)$
- Implication : Le taux de succès ne se dégrade jamais lorsque la taille du cache augmente. FIFO, en raison de l'anomalie de Bélády, n'appartient pas à cette classe.

Choix des Algorithmes en Pratique

La sélection d'un algorithme de cache dépend fortement du compromis entre le coût de l'algorithme et le coût d'un échec de cache.

- **Systèmes de Haut Niveau (Serveurs Web, Bases de Données)** : Le coût d'un échec (ex: recharger un fichier de 500 Mo depuis le disque) est très élevé. On est donc prêt à utiliser des algorithmes plus complexes et coûteux en calcul (comme LRU) pour maximiser le taux de succès.
- **Caches de Bas Niveau (Cache CPU)** : La gestion du cache est exécutée à chaque accès mémoire, elle doit donc avoir une surcharge extrêmement faible. Des algorithmes plus simples sont utilisés :
 - **Random** : Retire un objet au hasard. Très peu coûteux à implémenter et fonctionne raisonnablement bien pour les grands caches. Il est utilisé dans certains caches CPU.
 - **Approximations de LRU** : Des variantes modifiées de LRU sont utilisées pour réduire le coût de mise à jour à chaque accès.
 - **FIFO pur** est rarement utilisé en raison de ses faibles performances.

6.7.Résumé Comparatif des Technologies de Mémoire

Les tableaux suivants résument les caractéristiques des différentes technologies de mémoire abordées, illustrant les compromis qui justifient l'existence des hiérarchies de cache.

Tableau 1 : Cache CPU (SRAM) vs. Mémoire Principale (DRAM)

Caractéristique	SRAM	DRAM
Usage	Cache CPU	Mémoire Principale
Persistance	Volatile	Volatile
Latence	2–10 ns	50–100 ns
Débit	1–2 TB/s	0,5–12,8 GB/s
Taille typique	< 20 MB	4 GB – 256 GB
Prix/GB	> 1000 €	5 €

Tableau 2 : Cache Disque (DRAM) vs. Stockage Permanent (Disque Dur / SSD)

Caractéristique	DRAM	Disque Dur ou SSD
Usage	Cache Fichier	Stockage Permanent
Latence	50–100 ns	1–10 ms (HDD) / 100 μ s (SSD)
Débit	0,5–12,8 GB/s	100–550 MB/s (HDD) / jusqu'à 3 GB/s (SSD)
Taille typique	$\leq$ 256 GB	$\geq$ 1 TB
Prix/GB	5 €	0,05 €

7.Caches CPU : Partie 1

7.1.Résumé Exécutif

Ce document de synthèse analyse l'organisation, les principes et les architectures des caches de processeur (CPU), des composants critiques qui servent de pont entre la vitesse de calcul du processeur et la latence de la mémoire principale (RAM). L'objectif fondamental d'un cache est de stocker les données fréquemment utilisées pour minimiser le temps d'accès, mais sa conception est un exercice d'équilibre entre vitesse, coût et capacité.

Les points clés sont les suivants :

- **Optimisation par Blocs** : Les caches modernes n'opèrent pas sur des octets individuels mais sur des blocs de données de taille fixe, appelés "lignes de cache" (typiquement 64 octets). Cette approche exploite la **localité spatiale** des programmes et permet des **accès en rafale** à la RAM, améliorant considérablement l'efficacité des transferts de données.
- **Architectures Fondamentales** : Trois principaux modèles d'organisation définissent où un bloc de données peut être stocké :
 1. **Entièrement Associatif (Fully Associative)** : Offre une flexibilité maximale (un bloc peut aller n'importe où) et une recherche rapide via des comparateurs parallèles, mais son coût, sa consommation d'énergie et son encombrement le limitent à de très petits caches spécialisés.
 2. **À Correspondance Directe (Direct-Mapped)** : Chaque bloc mémoire a un emplacement unique et prédéterminé dans le cache. Cette simplicité le rend rapide et peu coûteux, mais il est vulnérable aux "échecs de conflit" lorsque plusieurs blocs de données actifs se disputent le même emplacement.
 3. **Associatif par Ensemble (Set-Associative)** : Le compromis le plus répandu en pratique. Le cache est divisé en ensembles, et chaque bloc mémoire est assigné à un ensemble spécifique mais peut occuper n'importe laquelle des quelques places ("voies") disponibles dans cet ensemble. Cette approche réduit considérablement les conflits tout en maintenant une excellente performance.
- **Hiérarchie Multi-niveaux** : Les processeurs modernes intègrent une hiérarchie de caches (L1, L2, L3) pour optimiser les performances à différents niveaux. Le cache L1 est le plus petit et le plus rapide, dédié à un seul cœur de processeur. Le L2 est plus grand et un peu plus lent. Le L3 est encore plus grand, plus lent et partagé entre tous les cœurs, nécessitant une associativité élevée pour gérer les accès concurrents.
- **Gestion du Contenu** : Lorsque le cache est plein, une **politique de remplacement** (ou d'éviction) décide quel bloc supprimer pour faire de la place. Bien que des algorithmes comme LRU (Least Recently Used) soient idéaux, les caches réels utilisent des approximations efficaces comme **Tree-Pseudo-LRU** ou des politiques plus simples (Aléatoire, FIFO) pour maintenir une vitesse maximale.

En somme, la conception des caches CPU est une science de compromis sophistiquée visant à maximiser le taux de succès ("hit rate") tout en respectant des contraintes physiques et économiques strictes, ce qui est essentiel pour les performances globales des systèmes informatiques modernes.

7.2.Fondamentaux des Caches CPU

Rôle et Exigences

Les caches sont des composants omniprésents dans les systèmes informatiques, servant de mémoire tampon rapide entre le processeur et la mémoire principale (RAM). Leurs contraintes de conception sont dictées par leur rôle critique.

- **Vitesse** : Les caches CPU doivent être extrêmement rapides, car chaque accès à la mémoire par le processeur passe d'abord par eux. Cela exclut l'utilisation de structures de données complexes comme les tables de hachage, qui introduiraient une latence inacceptable. Des structures de données et des algorithmes efficaces sont impératifs.
- **Coût et Taille** : Intégrés directement sur la puce du processeur, les caches sont très coûteux à produire. Par conséquent, leur taille est limitée, allant de quelques kilo-octets (Ko) à quelques méga-octets (Mo).

Une implémentation naïve qui stockerait des octets individuels avec leur adresse complète serait inefficace, car la majorité des circuits électroniques du cache serait utilisée pour stocker les identifiants (adresses) plutôt que les données elles-mêmes.

Optimisation par Lignes de Cache (Cachelines)

Pour surmonter les limitations de l'approche naïve, les caches modernes organisent les données en blocs de taille fixe, appelés **lignes de cache** (ou *cachelines*).

- **Principe** : Le cache ne stocke pas des octets individuels mais des blocs contigus de mémoire de taille b . Les tailles typiques pour b sont de 16, 64 ou 128 octets.
- **Avantage 1 - Localité Spatiale** : Cette organisation améliore significativement le taux de succès (*hit rate*). En vertu du principe de localité spatiale, si un programme accède à une adresse mémoire, il est très probable qu'il accède bientôt à des adresses voisines. En chargeant un bloc entier, le cache anticipe ces futurs accès.
- **Avantage 2 - Accès en Rafale (Burst Access)** : Les puces de mémoire modernes (DRAM) sont optimisées pour les accès en rafale. Transférer un bloc entier de 64 octets prend beaucoup moins de temps que d'effectuer deux accès distincts d'un octet à des adresses aléatoires. Cela est dû au fait que la préparation interne des circuits de la mémoire n'est faite qu'une seule fois pour tout le bloc.

Utilisation d'Étiquettes (Tags) au lieu d'Adresses Complètes

Pour économiser de l'espace, le cache ne stocke pas l'adresse mémoire complète de chaque bloc. Pour un bloc de taille $b = 2^n$ octets, les n bits de poids faible de l'adresse de début du bloc sont toujours nuls. Il est donc inutile de les stocker. L'adresse est décomposée :

- **Étiquette (Tag)** : Les bits de poids fort de l'adresse, qui identifient de manière unique le bloc mémoire.
- **Décalage (Offset)** : Les n bits de poids faible, qui indiquent la position de l'octet désiré à l'intérieur du bloc.

Partie de l'Adresse	Rôle
Étiquette (Tag)	Identifie le bloc de données stocké dans la ligne de cache.
Décalage (Offset)	Localise l'octet spécifique à l'intérieur de la ligne de cache.

7.3. Architectures d'Organisation du Cache

L'organisation interne du cache détermine où un bloc de données peut être placé et comment il peut être retrouvé.

Le Cache Entièrement Associatif (Fully Associative)

Dans cette architecture, un bloc de données provenant de la mémoire principale peut être placé dans n'importe quel emplacement disponible du cache.

- **Fonctionnement** : Pour trouver une donnée, le processeur compare l'étiquette de l'adresse recherchée avec les étiquettes de *toutes* les lignes de cache simultanément, grâce à un comparateur par ligne.
- **Avantages** : Le temps d'accès est constant ($O(1)$) et très rapide. Cette flexibilité élimine les échecs de cache liés à des conflits d'emplacement.
- **Inconvénients** : Cette architecture est très coûteuse en termes d'espace sur la puce, consomme beaucoup d'énergie (tous les comparateurs sont actifs en même temps) et est complexe à fabriquer.
- **Usage** : Réservée aux très petits caches où la performance est la priorité absolue.

Le Cache à Correspondance Directe (Direct-Mapped)

À l'opposé du modèle entièrement associatif, chaque bloc de la mémoire principale ne peut être stocké qu'à un seul et unique emplacement dans le cache.

- **Fonctionnement** : L'adresse mémoire est divisée en trois parties : Étiquette (Tag), Index de Ligne (Row Index) et Décalage (Offset). L'index détermine de manière unique la ligne du cache où le bloc doit se trouver. Une seule comparaison d'étiquette est nécessaire, sur cette ligne spécifique.
- **Avantages** : Simple, rapide et économique, car il ne nécessite qu'un seul circuit comparateur.
- **Inconvénients** : Le principal défaut est la **contention**. Si un programme accède de manière répétée à deux blocs de données différents qui ont le même index de ligne, ces blocs s'évinceront mutuellement du cache, provoquant des échecs de cache appelés **échecs de conflit** (*conflict misses*), même si le reste du cache est vide.

Le Cache Associatif par Ensemble (Set-Associative)

Cette architecture est un compromis efficace entre les deux extrêmes précédents et constitue la solution la plus utilisée dans les processeurs modernes.

- **Principe** : Le cache est divisé en **ensembles** (*sets*), qui correspondent aux lignes. Chaque ensemble peut contenir plusieurs blocs, appelés **voies** (*ways*). Un bloc mémoire est mappé à un ensemble spécifique via son index, mais il peut être placé dans n'importe quelle voie disponible au sein de cet ensemble.
- **Terminologie** : Un cache avec c voies par ensemble est dit **associatif par ensemble à c voies** (*c-way set-associative*).
- **Fonctionnement** : L'index de l'adresse sélectionne un ensemble, puis c comparateurs vérifient en parallèle les étiquettes des c voies de cet ensemble.

- **Avantages :**
 - Presque aussi rapide que la correspondance directe.
 - Réduit considérablement la probabilité de contention, car il existe c emplacements possibles pour les blocs ayant le même index.
- **Inconvénients :** Nécessite plus d'électronique (c comparateurs) que le cache à correspondance directe.
- **Cas Limites :** Un cache associatif à 1 voie ($c=1$) est équivalent à un cache à correspondance directe. Un cache avec un seul ensemble est équivalent à un cache entièrement associatif.

7.4. Hiérarchie des Caches dans les CPU Modernes

Les processeurs actuels utilisent plusieurs niveaux de cache pour créer une hiérarchie de mémoire qui équilibre vitesse, capacité et coût.

Niveaux de Cache (L1, L2, L3)

- **Cache L1 :** Le plus petit (quelques Ko), le plus rapide et le plus cher. Chaque cœur de processeur possède son propre cache L1, qui est souvent divisé en deux parties : un cache pour les instructions (i-cache) et un pour les données (d-cache). L'associativité est typiquement de 4 à 8 voies.
 - *Temps d'accès typique :* ~ 4 cycles d'horloge.
- **Cache L2 :** De taille intermédiaire (quelques centaines de Ko), plus lent que le L1 mais plus rapide que la RAM. Il est unifié (instructions et données) et généralement dédié à un seul cœur. L'associativité est également de 4 à 8 voies.
 - *Temps d'accès typique :* ~ 10 cycles d'horloge.
- **Cache L3 :** Le plus grand (2 à 8 Mo ou plus) et le plus lent des caches sur la puce. Il est partagé entre tous les cœurs du processeur. Son rôle est de capturer les données qui ne rentrent pas dans les caches L2 et de faciliter la communication entre les cœurs.
 - *Associativité :* Il possède une associativité élevée (12 voies ou plus) car, étant partagé, la probabilité que des adresses provenant de différents cœurs aient le même index est plus grande. Une haute associativité est cruciale pour minimiser les échecs de conflit.
 - *Temps d'accès typique :* 40-75 cycles d'horloge.

Politiques d'Inclusion vs. Exclusion

La gestion des données entre les niveaux de cache peut suivre deux logiques :

- **Inclusive :** Toute donnée présente dans le cache L1 est également présente dans le cache L2.
- **Exclusive :** Les données ne sont présentes que dans un seul niveau de cache à la fois. Si une donnée est déplacée du L2 vers le L1, elle est retirée du L2. Cette approche évite la duplication de données et maximise la capacité de stockage totale, mais engendre une gestion plus complexe.

Hiérarchie de Latence et Capacité

Le tableau suivant illustre l'échelle des ordres de grandeur pour la capacité et la latence à travers la hiérarchie de la mémoire.

Niveau de Mémoire	Capacité Typique	Latence Typique
Registre CPU	~128 octets	0.2 ns
Cache L1	32 Ko	~1 ns (0.5 ns)
Cache L2	256 Ko	~5 ns (7 ns)
RAM	8 Go	100 ns
Disque dur local (SSD)	1 To	~1-10 ms (150 µs)
Serveur distant	x To	> 500 ms
Sauvegarde sur bande	x To	> 5 minutes

7.5.Gestion du Contenu et Performance

Types d'Échecs de Cache (Cache Misses)

Un échec de cache (*cache miss*) se produit lorsque le processeur demande une donnée qui n'est pas dans le cache. On distingue plusieurs types d'échecs :

1. **Échecs de Démarrage (Warm-up)** : Se produisent au début de l'exécution d'un programme, lorsque le cache est encore vide.
2. **Échecs de Capacité** : Surviennent parce que le cache n'est pas assez grand pour contenir toutes les données nécessaires au programme à un instant T.
3. **Échecs de Conflit** : Spécifiques aux caches non-entièrement associatifs (direct-mapped et set-associative), ils se produisent lorsque trop de blocs de données se disputent le même ensemble.
4. **Échecs sur Objets non Cachables** : Certaines zones mémoire (par exemple, pour les E/S) ou certains types de données (contenu web dynamique) ne sont pas destinés à être mis en cache.

Politiques de Remplacement (Eviction Policies)

Quand un nouvel élément doit être chargé dans un ensemble de cache qui est déjà plein, une **politique de remplacement** doit décider quel bloc existant doit être évincé.

- **Politiques Courantes** : Les algorithmes classiques incluent Random, FIFO (First-In, First-Out), LRU (Least Recently Used) et MRU (Most Recently Used).
- **Utilisation en Pratique** : Les fabricants de CPU ne publient pas les détails de leurs implémentations. Cependant, on observe les tendances suivantes :
 - **Caches L1/L2** : La vitesse est primordiale. Des approximations de LRU comme **Tree-Pseudo-LRU** sont souvent utilisées. C'est un algorithme qui utilise un arbre binaire pour approximer le bloc le moins récemment utilisé avec très peu de bits de stockage. D'autres politiques simples comme Random, FIFO ou Round-Robin sont également employées.
 - **Cache L3** : Étant partagé, des politiques plus sophistiquées peuvent être bénéfiques. On y trouve des approximations de LRU, mais aussi des politiques MRU (pour conserver plus longtemps les vieilles données) ou des approches adaptatives qui mélangent différentes stratégies.

7.6.Exemples Concrets de Processeurs

Les caractéristiques des caches varient considérablement d'une génération de processeurs à l'autre, reflétant les progrès en matière de technologie et d'architecture.

Comparaison de Caches sur Processeurs Intel

Caractéristique	Intel Core i7-3610QM (Ivy Bridge)	Intel Core i5-1135G7 (Tiger Lake-U)
Cœurs/Threads	4 / 8	4 / 8
Cache L1 Data	4 x 32 KBytes, 8-way	4 x 48 KBytes, 12-way
Cache L1 Inst.	4 x 32 KBytes, 8-way	4 x 32 KBytes, 8-way
Cache L2	4 x 256 KBytes, 8-way	4 x 1.25 MBytes, 20-way
Cache L3	6 MBytes, 12-way	8 MBytes, 8-way

Cette comparaison montre que les processeurs plus récents tendent à avoir des caches plus grands et avec une associativité plus élevée pour améliorer les performances.

8.Caches CPU : Partie 2

8.1.Résumé Exécutif

Ce document synthétise les mécanismes avancés de gestion des caches CPU, en se concentrant sur les stratégies d'écriture, la cohérence des données dans les systèmes multi-cœurs, les techniques de pré-chargement (prefetching) et les principes de programmation orientée cache. Les opérations d'écriture sont gérées par deux politiques principales : **Write-Through (écriture simultanée)**, qui met à jour la mémoire principale et le cache simultanément, et **Write-Back (écriture différée)**, qui n'écrit que dans le cache et reporte la mise à jour de la mémoire au moment de l'éviction du bloc de cache. Cette dernière approche, plus complexe mais plus performante, utilise un "drapeau d'état" (Dirty flag) pour ne réécrire en mémoire que les blocs modifiés. Pour masquer la latence des écritures, les architectures modernes emploient des **tampons d'écriture (Write Buffers)** qui mettent en file d'attente les opérations destinées à la mémoire.

Dans les architectures multi-cœurs, le maintien de la **cohérence des caches** est crucial pour éviter les conflits de données. Le protocole de **"snooping"** est une solution où chaque cache surveille les actions des autres pour invalider ou mettre à jour ses propres copies de données partagées. Pour optimiser davantage les performances, le **prefetching** anticipe les besoins futurs du programme en chargeant les données dans le cache avant qu'elles ne soient explicitement demandées. Les mécanismes matériels comme le **Stream Prefetcher** (accès séquentiels) et le **Stride Prefetcher** (accès à intervalle constant) automatisent ce processus.

Enfin, l'efficacité du cache dépend fortement des pratiques de programmation. Le respect des principes de **localité spatiale et temporelle** est fondamental. Des techniques comme l'optimisation de l'ordre des boucles dans la multiplication de matrices, l'alignement des données et le regroupement des variables fréquemment utilisées ensemble permettent de maximiser le taux de succès du cache (hit rate) et de réduire considérablement les temps d'exécution.

8.2. Stratégies d'Écriture dans les Caches (Write Policies)

La gestion des opérations d'écriture est une composante critique de l'architecture des caches, car elle doit garantir la cohérence entre le cache, rapide mais volatil, et la mémoire principale, plus lente mais persistante. Deux stratégies principales existent, chacune avec ses propres compromis en termes de performance et de complexité.

Politique Write-Through (Écriture Simultanée)

Dans la stratégie Write-Through, les données sont écrites simultanément à deux endroits : directement dans la mémoire principale (backing store) et, si le bloc de mémoire affecté est déjà présent dans le cache, dans le cache également.

Lors d'une écriture sur une adresse mémoire qui n'est pas actuellement dans le cache (write miss), deux approches sont possibles :

- **Write-Allocate (ou Fetch-on-Write)** : Le bloc de mémoire concerné est d'abord chargé dans le cache, puis l'opération d'écriture est effectuée. Cette approche est motivée par le principe de localité : un programme qui écrit dans une zone de mémoire est susceptible de la lire peu de temps après.
- **Write-Around** : L'écriture est effectuée uniquement en mémoire principale, sans charger le bloc dans le cache. La motivation est que les programmes effectuent souvent plusieurs écritures successives dans une zone avant de la lire.

Le choix entre ces deux approches dépend du comportement du programme. Dans tous les cas, avec une politique Write-Through, le cache doit être mis à jour si le bloc est présent pour maintenir la cohérence.

Politique Write-Back (Écriture Différée)

La stratégie Write-Back est conçue pour maximiser les performances en minimisant les accès à la mémoire principale. Lorsqu'une écriture est effectuée, les données ne sont modifiées **que dans le cache**. La mise à jour de la mémoire principale est reportée jusqu'à ce que le bloc de cache concerné soit remplacé (évincé).

Pour gérer ce mécanisme, un **drapeau d'état "Dirty"** est associé à chaque bloc du cache.

- Ce drapeau est automatiquement activé lorsqu'une opération d'écriture modifie le bloc.
- Lorsqu'un bloc doit être remplacé, le système de cache vérifie ce drapeau. Si le drapeau "Dirty" est activé, le contenu du bloc est réécrit en mémoire principale avant que le nouveau bloc ne prenne sa place. Les blocs non modifiés sont simplement écrasés, évitant ainsi des écritures inutiles en mémoire.

Cette politique offre deux avantages majeurs :

1. **Vitesse** : Le CPU n'a pas à attendre la fin de l'écriture en mémoire principale, qui est beaucoup plus lente. Il peut continuer l'exécution dès que l'écriture dans le cache est terminée.
2. **Localité Spatiale** : Plusieurs écritures sur des adresses adjacentes situées dans le même bloc de cache (par exemple, x et $x+1$) n'entraînent qu'une seule opération d'écriture en mémoire lorsque le bloc est finalement évincé.

Optimisation avec les Write Buffers (Tampons d'Écriture)

Le remplacement d'un bloc "dirty" dans un cache Write-Back peut être lent, car il nécessite une écriture en mémoire suivie d'une lecture. Pour atténuer cette latence, les architectures modernes utilisent un **Write Buffer**.

- **Fonctionnement** : Il s'agit d'une file d'attente qui stocke les opérations d'écriture destinées à la mémoire. Le cache peut y placer un bloc "dirty" à réécrire et continuer immédiatement à charger le nouveau bloc demandé. L'écriture effective en mémoire est effectuée en arrière-plan lorsque le bus mémoire est disponible.
- **Applicabilité** : Bien que particulièrement utile pour les caches Write-Back, cette technique peut également être utilisée avec les caches Write-Through pour permettre au CPU de ne pas attendre la fin des écritures en mémoire.

- **Limites** : Si un programme effectue un grand nombre d'écritures en un court laps de temps, le tampon peut se remplir, forçant le CPU à attendre. En moyenne, les performances ne peuvent pas dépasser la vitesse de la mémoire principale.

8.3. Cohérence des Caches dans les Systèmes Multi-Cœurs

Avec l'avènement des processeurs multi-cœurs, où chaque cœur peut posséder son propre cache (par exemple, L1), un nouveau problème se pose : comment s'assurer que tous les cœurs voient une version cohérente des données en mémoire lorsque celles-ci sont partagées ?

Le Problème de la Consistance et le Protocole de "Snooping"

Si le cœur 1 modifie une donnée dans son cache L1, le cœur 2, qui pourrait avoir une ancienne copie de la même donnée dans son propre cache, risque de travailler avec une information obsolète. Pour résoudre ce problème, des **protocoles de cohérence de cache** sont mis en œuvre.

Une solution courante est le **"snooping"** (écoute) :

- Les caches surveillent en permanence les actions des autres caches sur le bus mémoire.
- Si un cache C1 modifie un bloc de données (le rendant "dirty") qui est également présent dans un autre cache C2, le protocole force C2 à **invalidier** (supprimer) sa copie.
- Si un cache C2 demande à lire un bloc de mémoire qui existe à l'état "dirty" dans C1, C1 doit d'abord **réécrire ce bloc en mémoire principale** (write back) avant que C2 ne puisse le lire, garantissant ainsi que C2 obtient la version la plus récente.

8.4. Prefetching : Anticiper les Besoins en Données

Le **prefetching** est une technique d'optimisation qui consiste à charger des données de la mémoire vers le cache **avant** qu'elles ne soient explicitement requises. L'objectif est de masquer la latence de l'accès à la mémoire. Cependant, une prédiction incorrecte peut dégrader les performances en gaspillant la bande passante et l'espace du cache.

Prefetching Logiciel et Matériel

- **Prefetching Logiciel** : Le programmeur ou le compilateur insère des instructions spéciales (`prefetch`) dans le code pour indiquer au CPU de charger à l'avance des données spécifiques.
- **Prefetching Matériel** : Le matériel du cache analyse les modèles d'accès à la mémoire pour prédire les accès futurs. Les implémentations courantes incluent :
 - **Stream Prefetcher** : Lorsqu'un bloc à l'adresse `x` est accédé, le prefetcher charge automatiquement le bloc suivant à l'adresse `x + taille_du_bloc`. C'est très efficace pour les opérations sur des tableaux, comme le traitement d'images.
 - **Stride Prefetcher** : Détecte les accès mémoire séparés par un intervalle constant (un "stride"), comme l'accès au même champ dans un tableau de structures. Une fois le stride identifié, il pré-charge les données des éléments suivants de la séquence.

8.5. Programmation Orientée Cache (Cache-Friendly Programming)

L'organisation du code et des données a un impact direct et significatif sur les performances du cache. Les programmeurs peuvent exploiter les principes de localité pour optimiser leurs applications.

Principes de Localité

- **Localité Temporelle** : Les éléments récemment accédés sont susceptibles d'être réutilisés prochainement. Les boucles sont un exemple classique.
- **Localité Spatiale** : Si un élément est accédé, les éléments aux adresses mémoire voisines sont susceptibles d'être accédés bientôt. Le parcours séquentiel d'un tableau en est l'illustration parfaite.

Techniques d'Optimisation Pratiques

- **Parcours de Matrice** : Une matrice étant stockée en mémoire ligne par ligne (row-major order), un parcours par lignes (`for i... for j... a[i][j]`) bénéficie d'une excellente localité spatiale et maximise les "cache hits". Un parcours par colonnes (`for j... for i... a[i][j]`) provoque des sauts importants en mémoire, entraînant un "cache miss" à presque chaque accès.
- **Multiplication de Matrices** : L'ordre des boucles (`ijk`, `ikj`, `jki`, etc.) change radicalement le modèle d'accès. La version `ijk` standard souffre d'un mauvais accès (par colonne) à la matrice B. Une version `ikj` est souvent plus performante, car elle maximise la réutilisation des éléments de A et B dans les boucles internes.
- **Organisation des Données** :
 - **Ordre des variables** : Placer les variables scalaires fréquemment utilisées ensemble dans une déclaration peut les faire résider dans la même ligne de cache, réduisant le nombre de lignes "dirty" à réécrire.
 - **Alignement des données** : Les compilateurs ajoutent souvent des octets de remplissage (padding) pour aligner le début des structures de données sur les frontières des lignes de cache, évitant qu'une structure ne chevauche deux lignes de cache.
 - **Structure de données** : Le choix entre un tableau de structures (AoS) et une structure de tableaux (SoA) doit correspondre au modèle d'accès du programme pour aider le prefetcher et améliorer la localité.

8.6. Architecture des Caches : Exemples Réels

Les caractéristiques des caches varient considérablement entre les différentes architectures de processeurs. Le tableau suivant compare les hiérarchies de cache de l'ARM Cortex-A53 et de l'Intel Core i7.

Caractéristique	ARM Cortex-A53	Intel Core i7
Cache L1 Organisation	Caches d'instructions et de données séparés	Caches d'instructions et de données séparés par cœur

Cache L1 Taille	- Configurable, 16 à 64 KiB chacun	32 KiB chacun pour instructions/données
Cache L1 Associativité	- 2-voies (I), 4-voies (D)	4-voies (I), 8-voies (D)
Cache L1 Remplacement	- Aléatoire (Random)	Approximated LRU
Cache L1 Taille de Bloc	- 64 octets	64 octets
Cache L1 Politique d'Écriture	- Write-back, politiques d'allocation variables (défaut : Write-allocate)	Write-back, No-write-allocate
Cache L1 Temps d'accès	- 2 cycles d'horloge	4 cycles d'horloge, pipeliné
Cache L2 Organisation	- Unifié (instructions et données)	Unifié (instructions et données) par cœur
Cache L2 Taille	- 128 KiB à 2 MiB	256 KiB (0.25 MiB)
Cache L2 Associativité	- 16-voies	8-voies
Cache L2 Remplacement	- Approximated LRU	Approximated LRU
Cache L2 Taille de Bloc	- 64 octets	64 octets
Cache L2 Politique d'Écriture	- Write-back, Write-allocate	Write-back, Write-allocate
Cache L2 Temps d'accès	- 12 cycles d'horloge	10 cycles d'horloge
Cache L3 Organisation	-	Unifié (instructions et données), partagé
Cache L3 Taille	-	8 MiB
Cache L3 Associativité	-	16-voies
Cache L3 Remplacement	-	Approximated LRU
Cache L3 Taille de Bloc	-	64 octets
Cache L3 Politique d'Écriture	-	Write-back, Write-allocate
Cache L3 Temps d'accès	-	35 cycles d'horloge

9. Outils de Débogage

9.1. Résumé Exécutif

Ce document présente une analyse approfondie d'une suite d'outils essentiels pour le débogage et l'analyse de performance des applications. L'objectif est d'identifier les goulots d'étranglement, de détecter les erreurs de gestion de la mémoire et de comprendre le comportement d'un programme en cours d'exécution. Les outils clés examinés sont :

- **Valgrind** : Un détecteur d'erreurs mémoire qui identifie les fuites, les doubles libérations (`double free`) et les accès hors limites (`out of bounds`), même en l'absence de `segmentation fault`.
- **GDB (GNU Debugger)** : Un débogueur interactif qui permet d'inspecter un programme en cours d'exécution, de définir des points d'arrêt, d'examiner les variables et les registres, et de localiser précisément la source des erreurs.
- **Perf** : Un puissant outil de profilage pour Linux qui utilise les compteurs de performance matériels du CPU. `perf stat` fournit des statistiques globales sur l'exécution, tandis que `perf record` et `perf report` permettent une analyse fine pour localiser les "points chauds" (fonctions ou lignes de code consommant le plus de ressources).
- **Cachegrind** : Un simulateur de cache précis, intégré à Valgrind, qui analyse le comportement du cache L1 et du cache de dernier niveau (LL). Il est particulièrement utile lorsque les compteurs matériels sont absents, mais son exécution est très lente. `callgrind`, une extension, offre des visualisations via `KCachegrind`.
- **Strace** : Un utilitaire qui intercepte et enregistre les appels système (`syscalls`) et les signaux d'un processus, offrant une visibilité sur les interactions du programme avec le noyau.
- **Intel Vtune Profiler** : Un profileur graphique avancé pour les processeurs Intel, agissant comme une surcouche conviviale à `perf`. Il fournit des analyses de microarchitecture, de mémoire et de parallélisme.

Ensemble, ces outils forment un arsenal complet pour optimiser et fiabiliser les logiciels, allant de la chasse aux erreurs mémoire bas niveau à l'analyse de performance à l'échelle du système.

9.2. Analyse Détaillée des Outils

Valgrind : Détection d'Erreurs de Gestion de la Mémoire

Valgrind est un outil fondamental pour assurer la robustesse d'une application en C/C++. Il suit chaque allocation (`malloc`) et libération (`free`) de mémoire pour détecter des anomalies critiques.

Fonctionnalités Clés :

- **Détection de fuites de mémoire** : Identifie les blocs de mémoire alloués mais jamais libérés.
- **Détection de double libération** : Repère les tentatives de libérer un même bloc de mémoire plus d'une fois.

- **Détection d'accès hors limites** : Signale les lectures ou écritures en dehors des bornes d'un bloc de mémoire alloué, une cause fréquente de corruption de données et de plantages.

Exemples d'Utilisation :

- **Allocation implicite** : Même un programme simple comme "Hello world" utilisant `printf` peut allouer de la mémoire (1024 octets dans l'exemple), ce que Valgrind révèle. L'outil `massif` de Valgrind peut tracer ces allocations.
- **Localisation des fuites** :
 - Lorsqu'un `malloc` n'est pas suivi d'un `free`, Valgrind rapporte précisément le nombre d'octets "définitivement perdus" (`definitely lost`).
 - L'utilisation de l'option `--leak-check=full` et la compilation du programme avec les symboles de débogage (`-g` ou `-gdwarf`) permettent à Valgrind d'indiquer la ligne exacte du code source où la mémoire a été allouée.
- **Détection de double free et out of bounds** : Valgrind intercepte ces erreurs à l'exécution et fournit une trace complète, indiquant l'adresse de l'accès invalide, le bloc de mémoire concerné, et les lignes de code où le bloc a été alloué et où l'erreur s'est produite. Il est capable de détecter un accès hors limites même si celui-ci ne provoque pas de `segmentation fault` immédiat.

GDB : Débogueur Interactif

GDB est l'outil de choix pour analyser le comportement d'un programme en cours d'exécution de manière interactive. Il permet de "geler" l'exécution pour inspecter son état à un instant T.

Fonctionnalités Clés :

- **Lancement et interruption** :
 - On lance un programme sous GDB avec `gdb --args ./programme [arguments]`.
 - L'exécution peut être interrompue à tout moment avec `CTRL + C` (signal `SIGINT`).
- **Points d'arrêt (Breakpoints)** : La commande `break nom_de_la_fonction` met en pause l'exécution juste avant l'appel de la fonction spécifiée.
- **Inspection de l'état** :
 - `where` : Affiche la pile d'appels (`stack trace`), montrant la séquence de fonctions qui a mené au point d'arrêt.
 - `info variables / info locals` : Liste les variables globales ou locales et leurs valeurs.
 - `display nom_variable` : Affiche la valeur d'une variable à chaque pas d'exécution.
 - `info registers` : Affiche le contenu des registres généraux du CPU.
 - `info all-registers` : Inclut également les registres spécialisés (SSE, virgule flottante, etc.).
- **Gestion des erreurs** : En cas de plantage (ex: `SIGSEGV`, `Segmentation fault`), GDB s'arrête automatiquement sur la ligne de code exacte qui a provoqué l'erreur, permettant une analyse immédiate avec `where`.

Perf : Analyse de Performance et des Goulots d'Étranglement

`perf` est un outil de profilage système qui s'appuie sur les compteurs de performance matériels pour mesurer divers événements (cycles CPU, accès cache, etc.) avec une faible surcharge.

`perf stat` : Statistiques Globales

`perf stat` exécute une commande et rapporte les comptes totaux pour plusieurs événements de performance.

- **Événements par défaut :**
 - `cpu-cycles` : Nombre de cycles CPU consommés.
 - `instructions` : Nombre d'instructions exécutées (le ratio `insn per cycle` est un indicateur d'efficacité).
 - `branch-misses` : Pourcentage de prédictions de branchement erronées, qui pénalisent la performance.
- **Utilisation avancée :**
 - **Contrôle temporel** : Les options `--timeout` (durée) et `-D` (délai) sont cruciales pour mesurer uniquement la période de charge pertinente d'une application (par exemple, un serveur web sous le feu d'un outil comme `wrk`).
 - **Événements spécifiques** : La commande `perf list` révèle des centaines d'événements matériels. On peut les spécifier avec l'option `-e`, par exemple pour mesurer précisément les ratés des caches de données L1 (`L1-dcache-load-misses`), du cache de dernier niveau (`LLC-misses`), ou des TLB (`dTLB-load-misses`).

`perf record` & `perf report` : Profilage Détaillé

Ce duo permet de savoir *où* dans le code les événements de performance se produisent.

- **`perf record` :**
 - Enregistre des échantillons à haute fréquence (ex: 999 fois par seconde avec `-F 999`).
 - À chaque échantillon, il note l'instruction en cours d'exécution et la pile d'appels (`--call-graph dwarf`).
 - Cela permet de corréler les événements (ex: cycles CPU) avec des fonctions ou des lignes de code spécifiques.
- **`perf report` :**
 - Analyse le fichier de données généré par `perf record`.
 - Présente une vue interactive où les fonctions sont classées par "poids" (pourcentage de temps CPU ou d'événements).
 - Permet de naviguer dans le code pour voir la répartition des coûts au niveau des instructions assembleur.
- **Importance des symboles de débogage :**
 - Sans symboles de débogage, `perf report` n'affiche que le code assembleur, difficile à mapper au code source.
 - Compiler avec `-gdwarf` permet à `perf report` d'annoter le code assembleur avec les lignes de code C correspondantes.

- Un compromis doit être trouvé pour le niveau d'optimisation (`-O1`, `-O3`) : moins d'optimisation facilite la lecture mais peut ne pas refléter les conditions de performance réelles.

Cachegrind : Simulation de Cache Détaillée

Cachegrind est un outil de Valgrind qui simule le comportement des caches du processeur.

- **Objectif** : Mesurer les références et les ratés des caches d'instructions (I1) et de données (D1), ainsi que du cache unifié de dernier niveau (LL). Il est utile car certains processeurs (notamment Intel) ne fournissent pas de compteurs matériels pour tous les types de ratés (ex: `L1i misses`).
- **Inconvénients** : La simulation est extrêmement lente comparée à l'utilisation des compteurs matériels par `perf`. Les résultats sont précis mais basés sur un modèle de cache, pas sur le matériel réel.
- **Callgrind & KCachegrind** : `callgrind` est une extension qui, en plus de la simulation de cache (`--simulate-cache=yes`), collecte des données sur les graphes d'appels. L'outil graphique KCachegrind permet ensuite de visualiser ces données de manière intuitive, montrant quelles fonctions génèrent le plus de ratés de cache.

Outils Complémentaires

Strace

- **Fonction** : Affiche tous les appels système et signaux échangés entre un processus et le noyau.
- **Utilité** : Très efficace pour déboguer des problèmes d'interaction avec le système d'exploitation (accès fichiers, opérations réseau, permissions). La commande `strace -f` permet de suivre également les processus enfants.

Intel Vtune Profiler

- **Description** : Un profileur de performance professionnel avec une interface graphique, présenté comme une alternative plus "facile à utiliser" que `perf` pour les machines Intel.
- **Capacités** :
 - Permet le profilage à distance via SSH.
 - Génère des rapports synthétiques sur la performance (Performance Snapshot) qui identifient les facteurs limitants : microarchitecture (pipeline CPU), accès mémoire (`Memory Bound`), ou vectorisation.
- **Prérequis** : Nécessite une machine équipée d'un processeur Intel et des privilèges `sudo` pour configurer les permissions de traçage du noyau. Bien que puissant, son utilisation n'est pas un attendu principal dans le cadre du projet décrit.

10.Parallélisme au niveau des instructions et Pipelining du CPU

10.1.Résumé analytique

Ce document de synthèse analyse les stratégies fondamentales utilisées dans les processeurs (CPU) modernes pour atteindre un haut niveau de performance grâce au parallélisme au niveau des instructions (Instruction-Level Parallelism - ILP). La technique centrale est le **pipelining**, qui décompose l'exécution d'une instruction en plusieurs étapes se déroulant en parallèle pour différentes instructions, augmentant ainsi considérablement le débit.

Cependant, l'efficacité du pipeline est menacée par des **verrouillages (stalls)**, principalement causés par des dépendances de données (un calcul nécessitant le résultat d'un autre non terminé) et des instructions de branchement (sauts conditionnels et inconditionnels) qui rompent le flux séquentiel. Pour surmonter ces obstacles, les architectures de CPU ont évolué pour inclure des mécanismes sophistiqués :

1. **Exécution dans le désordre (Out-of-Order Execution)** : Les CPU réorganisent dynamiquement les instructions pendant l'exécution pour traiter des opérations indépendantes pendant que d'autres sont en attente, minimisant ainsi les temps morts.
2. **Exécution spéculative et prédiction de branchement** : Pour gérer les sauts, les CPU prédisent l'issue d'un branchement et exécutent à l'avance les instructions du chemin le plus probable. Des prédicteurs de branchement avancés atteignent des taux de réussite supérieurs à 97 %, rendant cette technique très efficace malgré la pénalité d'une prédiction erronée.
3. **Superscalarité** : Les processeurs intègrent plusieurs unités d'exécution (ALU, FPU, etc.) pour traiter plusieurs instructions totalement en parallèle à chaque cycle d'horloge.

Ces techniques sont complétées par des mécanismes comme le **renommage de registres**, qui élimine les fausses dépendances de données, et la décomposition d'instructions complexes en **micro-opérations (μOPs)** plus simples. En conclusion, un CPU moderne fonctionne comme une machine virtuelle, traduisant les instructions du programme en un flux interne de micro-opérations optimisées pour une exécution parallèle massive, une complexité nécessaire car les gains de performance par l'augmentation de la fréquence d'horloge ont atteint leurs limites physiques et économiques.

10.2.Le Pipelining d'instructions : Le fondement de la performance

Le pipelining est une technique universelle dans la conception des processeurs depuis 1985, exploitant le parallélisme au niveau des instructions. Plutôt que d'exécuter une instruction de bout en bout avant de commencer la suivante, le processus est divisé en plusieurs étapes distinctes qui peuvent être exécutées en parallèle pour différentes instructions.

Étapes typiques d'un pipeline d'instruction

L'exécution d'une seule instruction est une tâche complexe qui peut être décomposée comme suit :

Étape	Description	Durée typique (en cycles)
1. Fetch	Récupérer l'instruction depuis la mémoire (cache L1).	<1 (en moyenne grâce au prefetching)
2. Decode	Analyser et décoder l'instruction.	1
3. Read Operands	Lire le contenu des registres sources.	1
4. Perform Operation	Exécuter l'opération arithmétique ou logique (ALU).	1
5. Write Result	Écrire le résultat dans le registre de destination.	1
Total		~5 cycles

Grâce au pipeline, alors qu'une instruction est à l'étape d'exécution, la suivante est à l'étape de lecture des opérandes, celle d'après au décodage, etc. Idéalement, une nouvelle instruction peut ainsi se terminer à chaque cycle d'horloge, permettant à un CPU de 1 GHz d'exécuter bien plus d'instructions que ne le suggère le temps d'exécution total d'une seule instruction (5 cycles). La longueur des pipelines varie de 2 étages dans les anciens microcontrôleurs à entre 7 et 30 étages dans les CPU de bureau modernes (Intel, AMD, ARM).

10.3. Les obstacles du Pipeline : Verrouillages (Stalls) et dépendances

L'efficacité du pipeline dépend d'un flux continu d'instructions. Malheureusement, certaines situations, appelées "hazards" ou aléas, forcent le pipeline à s'arrêter, créant un "stall" (verrouillage).

- **Verrouillage dû à l'accès mémoire** : Si une instruction doit lire une donnée depuis la mémoire principale (`load 0x1000, r1`), le pipeline doit attendre plusieurs cycles que l'accès mémoire se termine avant de pouvoir continuer.
- **Verrouillage dû aux dépendances de données (Read After Write Hazard)** : C'est le cas le plus courant. Une instruction dépend du résultat d'une instruction précédente qui n'a pas encore terminé son exécution. Par exemple :
 - `I1: mult #2, r2`
 - `I2: add r2, r1` L'instruction I2 ne peut pas lire la valeur de `r2` avant que I1 n'ait fini son calcul et écrit le résultat. Le pipeline doit donc attendre.

10.4. Stratégies d'optimisation pour surmonter les obstacles

Pour minimiser l'impact de ces verrouillages, les compilateurs et les CPU utilisent une série de techniques d'optimisation. L'objectif est de minimiser le CPI (Cycles Per Instruction).

$$\text{CPI Pipeline} = \text{CPI Idéal} + \text{Stalls structurels} + \text{Stalls de données} + \text{Stalls de contrôle}$$

Réorganisation des instructions (Out-of-Order Execution)

Une méthode efficace pour combler les cycles perdus lors d'un stall est de réorganiser les instructions. Si une instruction indépendante suit une instruction qui provoque un stall, elle peut être exécutée avant.

- **Exemple :**

- Séquence originale avec stall : `mult #2, r2 (stall) add r2, r1 ; sub #1, r0`
- Séquence réorganisée : `mult #2, r2 ; sub #1, r0 (exécutée pendant le stall de mult) add r2, r1`

Cette réorganisation peut être effectuée **statiquement par le compilateur** ou, de manière plus puissante, **dynamiquement par le CPU** pendant l'exécution. C'est ce qu'on appelle l'exécution dans le désordre (*out-of-order execution*), où les instructions décodées sont placées dans un tampon et exécutées dès que leurs opérandes sont prêts.

Gestion des branchements (Jumps)

Les instructions de branchement (sauts conditionnels, inconditionnels, boucles, appels de fonction) sont particulièrement néfastes pour le pipeline car elles changent l'adresse de la prochaine instruction à exécuter. Le pipeline doit souvent être vidé et redémarré à la nouvelle adresse, ce qui peut coûter de 10 à 20 cycles sur un pipeline long.

Optimisations par le compilateur

- **Inlining de fonctions** : Pour les fonctions courtes ou peu appelées, le compilateur remplace l'appel de fonction directement par le code de la fonction, éliminant ainsi le saut. Cela augmente la taille du code, ce qui peut être un inconvénient pour le cache d'instructions.
- **Déroulage de boucle (Loop Unrolling)** : Le compilateur duplique le corps de la boucle pour réduire le nombre total d'itérations et donc le nombre d'instructions de comparaison et de saut. Pour les boucles avec un petit nombre connu d'itérations, la boucle peut être complètement déroulée.
- **Strip Mining** : Technique de déroulage pour les boucles dont le nombre d'itérations est inconnu à la compilation.

Optimisations matérielles

- **Exécution spéculative** : Au lieu d'attendre le résultat d'une comparaison pour un saut conditionnel, le CPU "prédit" l'issue (par exemple, que le saut ne sera pas pris) et continue l'exécution de manière spéculative. Si la prédiction est correcte, il n'y a aucune perte de performance. Si elle est incorrecte, le pipeline est vidé et redémarré, entraînant une pénalité significative.
- **Prédiction de branchement** : Pour que l'exécution spéculative soit efficace, la prédiction doit être précise. Les CPU modernes intègrent des prédicteurs de branchement très sophistiqués.
 - **Compteurs sur 2 bits** : Une table garde en mémoire pour chaque saut s'il a été pris ou non lors de ses dernières exécutions.
 - **Tables à deux niveaux** : Ces systèmes plus complexes détectent des schémas de branchement (ex: "pris, non pris, pris, non pris..."). Les prédicteurs modernes atteignent une précision supérieure à 97 %.

- **Prédicteur d'adresse de retour (Return Address Predictor)** : Les retours de fonction sont des sauts inconditionnels fréquents. Un tampon spécial organisé en pile permet de prédire efficacement l'adresse de retour, même lorsqu'une fonction est appelée depuis plusieurs endroits différents.

10.5. La Superscalarité : Exécuter plus en parallèle

Au-delà de l'optimisation du pipeline, les CPU modernes sont **superscalaires**, ce qui signifie qu'ils disposent de plusieurs unités d'exécution (plusieurs ALU, FPU, etc.) et peuvent donc exécuter plusieurs instructions indépendantes au cours du même cycle d'horloge. Un CPU Intel peut, dans des conditions idéales, exécuter jusqu'à 10 instructions par cycle et par cœur.

Pour maximiser l'efficacité de l'architecture superscalaire, d'autres techniques sont nécessaires.

Fusion d'instructions

Certaines paires d'instructions apparaissent très fréquemment, comme une comparaison suivie d'un saut conditionnel (`compare` puis `jumpIfGreater`). Au lieu de les traiter séparément, une étape de "fusion d'instructions" dans le pipeline les combine en une seule "super-instruction" interne, optimisant leur traitement.

Renommage de registres

Parfois, des instructions sont logiquement indépendantes mais utilisent le même registre, créant une fausse dépendance qui empêche leur exécution parallèle.

- `l1: add r1, r2`
- `l2: set #4, r1` `l2` semble dépendre de `l1` car il écrit dans `r1`. Le **renommage de registres** résout ce problème. Le CPU dispose d'un grand nombre de registres physiques internes, invisibles pour le programmeur. Il mappe de manière transparente les registres architecturaux (`r1`) sur différents registres physiques.
- `l1: add r1_physA, r2_physB`
- `l2: set #4, r1_physC` Les deux instructions peuvent maintenant s'exécuter en parallèle. C'est crucial pour les architectures comme x86 qui ont peu de registres architecturaux.

Micro-opérations (μOPs)

Dans les processeurs Intel, le pipeline est encore plus complexe. Après la fusion, les instructions x86 (qui peuvent être très complexes) sont décomposées en une série de **micro-opérations (μOPs)** beaucoup plus simples et uniformes. Ces μOPs sont ensuite réorganisées, renommées, et envoyées aux différentes unités d'exécution parallèles (appelées "ports"). Cette décomposition offre une granularité plus fine pour l'exécution parallèle et la réorganisation. Les diagrammes des microarchitectures Nehalem et Skylake illustrent cette immense complexité.

10.6. Le CPU moderne comme une machine virtuelle

L'ensemble de ces efforts vise un objectif unique : la **vitesse**, mesurée par le nombre d'instructions exécutées par seconde. L'augmentation de la fréquence d'horloge étant devenue

extrêmement difficile et coûteuse (plafonnant autour de 4 GHz pour le grand public) et la programmation multi-cœur restant complexe, l'exploitation du parallélisme au niveau des instructions est la clé de la performance des cœurs de CPU individuels.

Un CPU Intel ou AMD moderne peut être vu comme une machine virtuelle. Les instructions machines utilisées par les programmes ne sont pas celles qui sont réellement exécutées. Elles sont traduites en un flux interne de micro-opérations, qui est ensuite massivement réorganisé et parallélisé par le matériel pour extraire le maximum de performance possible du code.

11.Parallélisme au Niveau des Données et les Architectures GPU

11.1.Résumé Exécutif

Ce document de synthèse analyse les concepts fondamentaux et les applications du parallélisme dans l'architecture informatique moderne, en se basant exclusivement sur les contextes fournis. L'objectif principal de l'exploitation du parallélisme est d'améliorer les performances en utilisant simultanément plusieurs unités de calcul. Les principaux paradigmes de parallélisme abordés sont le **SIMD (Single Instruction, Multiple Data)**, où une seule instruction opère sur un vecteur de données, et le **MIMD (Multiple Instruction, Multiple Data)**, qui correspond aux architectures multi-cœurs où chaque cœur exécute son propre flux d'instructions. D'autres formes clés incluent le **SMT (Simultaneous MultiThreading)**, une technique matérielle pour masquer la latence mémoire en faisant apparaître un cœur physique comme deux cœurs logiques, et le **SIMT (Single Instruction, Multiple Threads)**, le modèle de base des GPU qui exécute la même instruction sur de nombreux threads.

La performance de la parallélisation est intrinsèquement limitée par la partie séquentielle d'un programme, un principe formalisé par la **Loi d'Amdahl**. L'amélioration potentielle dépend de la fraction du code qui peut être parallélisée. Dans les CPU modernes, le parallélisme au niveau des données est implémenté via des extensions de jeu d'instructions comme **SSE et AVX**, qui permettent des opérations sur des registres vectoriels de 128 à 512 bits. La programmation de ces extensions se fait via des fonctions intrinsèques en C, offrant un contrôle explicite sur les opérations vectorielles, bien que les compilateurs puissent également effectuer une auto-vectorisation avec plus ou moins de succès.

Les **GPU** représentent une approche radicalement différente, optimisée pour un parallélisme de données massif. Leur architecture privilégie un grand nombre d'unités de calcul simples (ALU) au détriment de caches complexes et d'unités de contrôle sophistiquées. Les GPU utilisent le modèle **SIMT** pour gérer des milliers de threads simultanément. Cette surabondance de threads permet de masquer efficacement la latence mémoire : lorsqu'un groupe de threads est bloqué en attente de données, le matériel bascule instantanément vers un autre groupe prêt à être exécuté. La performance des architectures parallèles peut être modélisée à l'aide du **diagramme Roofline**, qui caractérise les limites d'un système en fonction de son intensité arithmétique (FLOPs/Byte), distinguant les applications limitées par la bande passante mémoire de celles limitées par la puissance de calcul brute.

11.2.Formes de Parallélisme en Architecture Informatique

Le parallélisme peut être classifié selon la taxonomie de Flynn, qui distingue les architectures en fonction du nombre de flux d'instructions et de données qu'elles peuvent traiter simultanément.

	Flux de Données Unique (Single)	Flux de Données Multiples (Multiple)
Flux d'Instruction Unique (Single)	SISD : Single Instruction, Single Data (ex: Intel Pentium 4)	SIMD : Single Instruction, Multiple Data (ex: instructions SSE de x86)

Flux d'Instruction Multiple (Multiple)	MISD: Multiple Instruction, Single Data (cas d'usage très spécifique)	MIMD: Multiple Instruction, Multiple Data (ex: Intel Xeon e5345)
---	--	---

Au-delà de cette taxonomie classique, d'autres formes de parallélisme sont cruciales dans les systèmes modernes.

Types de Parallélisme

- **Parallélisme au niveau de l'instruction (ILP)** : Exécution de plusieurs instructions par cycle d'horloge.
- **Parallélisme de Données (Data-Level Parallelism - DLP)** : Une seule instruction opère sur un vecteur de données (SIMD).
- **Parallélisme de Tâches (Task Parallelism)** : Utilisation de plusieurs processeurs ou cœurs pour exécuter différentes tâches (MIMD). Si chaque cœur dispose également de capacités SIMD, on pourrait parler de "MIM²D", bien que ce terme ne soit pas utilisé.
- **Parallélisme de Threads (Thread-Level Parallelism - TLP)** :
 - **Simultaneous MultiThreading (SMT)** : Technique matérielle où un seul cœur physique exécute des instructions de plusieurs threads pour masquer les temps d'attente (stalls), notamment ceux liés à la mémoire. Pour le système d'exploitation, le cœur physique apparaît comme plusieurs cœurs "logiques". Le SMT duplique les registres et le pointeur d'instruction, mais partage la plupart des autres ressources comme l'ALU et les caches, ce qui peut créer de la contention entre les applications. La technologie HyperThreading d'Intel est un exemple commercial de SMT.
 - **Single Instruction, Multiple Threads (SIMT)** : Modèle principalement utilisé dans les GPU, où la même instruction est exécutée par de nombreux threads, chacun possédant ses propres registres. Ce modèle est confronté au défi de la "divergence de threads" lorsque des branchements conditionnels amènent les threads à suivre des chemins d'exécution différents.

11.3.Évaluation de la Performance et Limites Théoriques

L'évaluation de l'efficacité de la parallélisation repose sur des métriques et des lois fondamentales qui en définissent les limites.

Speedup (Accélération)

Le "Speedup" mesure le gain de performance obtenu grâce à une amélioration. Il est calculé comme le rapport entre la performance avec l'amélioration et la performance sans celle-ci.

$$\text{Speedup} = \frac{\text{Performance pour la tâche entière avec amélioration}}{\text{Performance pour la tâche entière sans amélioration}}$$

Loi d'Amdahl

La loi d'Amdahl stipule que l'accélération globale d'un programme est limitée par sa fraction séquentielle, c'est-à-dire la partie du code qui ne peut pas être parallélisée.

$$\text{Speedup} = \frac{1}{(1 - \text{Fraction améliorée}) + \frac{\text{Fraction améliorée}}{\text{Speedup de l'amélioration}}}$$

Exemples :

- Avec 10 cœurs (Speedup de l'amélioration = 10), si une application passe 60% de son temps dans une partie non parallélisable (Fraction améliorée = 0.4), le Speedup global est de seulement 1.56.
- Avec un moteur SIMD traitant 8 flottants à la fois (Speedup de l'amélioration = 8), si 60% du temps est consacré à des opérations non mathématiques (Fraction améliorée = 0.4), le Speedup est de 1.53.

Strong vs. Weak Scaling

- **Strong Scaling (Scalabilité Forte)** : Mesure la capacité d'un programme à s'exécuter plus rapidement sur un plus grand nombre de processeurs pour une **taille de problème fixe**. La loi d'Amdahl s'applique directement ici.
- **Weak Scaling (Scalabilité Faible)** : Mesure la capacité d'un programme à résoudre un problème plus grand dans le même laps de temps en augmentant le nombre de processeurs. La **taille du problème est proportionnelle au nombre de processeurs**. Cela peut donner l'impression de "dépasser" la loi d'Amdahl, mais ne rend pas plus rapide une tâche unique de taille fixe.

11.4.Parallélisme au Niveau des Données : SIMD

SIMD est une extension du jeu d'instructions (ISA) qui permet d'exploiter un parallélisme de données significatif. Il est particulièrement efficace pour le calcul scientifique matriciel et le traitement multimédia.

- **Avantages :**
 - **Efficacité énergétique** : Une seule instruction est récupérée (fetch) pour plusieurs opérations sur les données.
 - **Simplicité de programmation** : Le programmeur peut continuer à penser de manière séquentielle, sans avoir à gérer la synchronisation ou la coordination complexes requises par la programmation multi-threads.

Implémentations SIMD sur x86

Extension	Année	Largeur du Registre	Description
MMX	1996	64 bits	Opérations sur 8 entiers de 8 bits ou 4 de 16 bits.
SSE	1999	128 bits	Opérations sur 4 flottants 32 bits ou 2 doubles 64 bits.
AVX	2010	256 bits	Opérations sur 8 flottants 32 bits ou 4 doubles 64 bits.
AVX-512	2017	512 bits	Opérations sur 16 flottants 32 bits ou 8 doubles 64 bits.

Les registres correspondants sont nommés `xmm` (SSE), `ymm` (AVX) et `zmm` (AVX-512). Les opérandes pour les instructions SIMD doivent être des emplacements mémoire consécutifs et alignés pour une performance optimale.

Programmation SIMD avec des Intrinsèques en C

L'utilisation de SIMD en C/C++ se fait via des "intrinsèques", qui sont des fonctions mappées directement sur des instructions assembleur.

- **Chargement et Stockage** : Les données doivent être explicitement chargées depuis la mémoire vers les registres SIMD (`_mm256_load_pd`) et stockées en retour (`_mm256_store_pd`). Des versions non alignées (`_mm256_loadu_pd`) existent mais sont moins performantes.
- **Opérations** : Des intrinsèques existent pour toutes les opérations arithmétiques (`_mm256_add_pd` pour une addition), logiques et de comparaison.
- **Branchement Conditionnel** : Le branchement traditionnel n'est pas possible au milieu d'un vecteur. À la place, on utilise un mécanisme de masquage et de fusion ("blending") :
 1. Une instruction de comparaison (`_mm256_cmp_pd`) génère un masque de bits.
 2. Les deux résultats possibles du branchement sont calculés.
 3. Une instruction de "blend" (`_mm256_blendv_ps`) sélectionne les éléments de l'un ou l'autre des résultats en fonction du masque.

Auto-vectorisation par le Compilateur

Les compilateurs modernes (comme GCC) peuvent tenter de convertir automatiquement des boucles scalaires en code SIMD ("auto-vectorisation"). Cependant, cette optimisation n'est pas magique. Le compilateur peut générer du code sous-optimal en incluant des vérifications pour les cas non alignés ou les tailles de boucle non multiples de la largeur du vecteur ("strip-mining"). Le programmeur peut aider le compilateur avec des "hints" comme `__assume` pour spécifier des contraintes (par ex., la taille est un multiple de 4), mais la performance du code manuel reste souvent supérieure.

11.5. Architectures GPU et Parallélisme de Threads : SIMT

Les GPU (Graphics Processing Units) ont évolué de simples cartes graphiques à de puissants processeurs massivement parallèles, conçus pour le calcul à usage général (GPGPU).

Architecture CPU vs. GPU

- **CPU** : Conçu pour une faible latence sur des tâches séquentielles. Il dispose de peu de cœurs puissants, de grandes quantités de mémoire cache et de logiques de contrôle complexes (prédiction de branchement, exécution out-of-order).
- **GPU** : Conçu pour un débit élevé sur des tâches massivement parallèles. Il contient des centaines ou des milliers de cœurs plus simples (ALU), moins de cache, et une logique de contrôle simplifiée. Il utilise le multithreading massif pour masquer la latence mémoire.

Le Modèle SIMT et l'Exécution sur GPU (CUDA)

- **Hiérarchie d'Exécution** : Le code à exécuter sur un GPU, appelé "Kernel", est organisé en une grille de blocs de threads.
 - Un **thread** est associé à un élément de donnée.
 - Un **bloc** est un groupe de threads (par ex., 32 threads, aussi appelé "warp") exécutés ensemble sur un **Streaming Multiprocessor (SM)**.

- Le GPU possède un planificateur matériel (ex: NVIDIA GigaThread) qui distribue les blocs aux SM disponibles. Il n'y a pas d'OS sur un GPU.
- **Divergence de Threads** : Lorsque les threads d'un même bloc rencontrent un branchement conditionnel et prennent des chemins différents, il y a divergence. Le matériel gère cela en exécutant séquentiellement chaque chemin de branchement, en désactivant les threads qui ne suivent pas ce chemin via des masques internes. Cela a un coût en performance, il est donc souvent préférable d'éviter la divergence.
- **Modèle d'Exécution Host/Device** : Le CPU (host) gère l'exécution globale et lance des Kernels sur le GPU (device). Le flux typique est :
 1. Le CPU exécute du code séquentiel.
 2. Le CPU lance un Kernel sur le GPU.
 3. Le GPU exécute le Kernel en parallèle.
 4. Le CPU attend la fin, puis récupère les résultats pour continuer son exécution.

11.6. Modélisation de la Performance : Le Diagramme Roofline

Le diagramme Roofline est un modèle visuel qui caractérise les performances maximales atteignables d'un système en fonction des caractéristiques de l'application.

- **Axes du Diagramme** :
 - **Axe Y** : Performance atteignable, mesurée en GFLOPs/seconde (Giga Opérations Flottantes par Seconde).
 - **Axe X** : Intensité Arithmétique, mesurée en FLOPs/Byte (nombre d'opérations flottantes par octet de données transféré depuis la mémoire).
- **Les "Toits" (Limits)** :
 1. **Toit Horizontal (Peak FP Performance)** : La performance de calcul maximale du processeur. Une application ne peut pas aller plus vite que cette limite, quelle que soit son intensité arithmétique.
 2. **Toit Incliné (Peak Memory Bandwidth)** : La bande passante mémoire maximale du système. La performance des applications à faible intensité arithmétique est limitée par la vitesse à laquelle les données peuvent être lues/écrites en mémoire.

$$\text{Performance Atteignable} = \min(\text{Peak FP Performance}, \text{Peak Memory BW} \times \text{Intensité Arithmétique})$$

Ce modèle permet d'identifier si un noyau de calcul est **limité par le calcul (Computation limited)** ou **limité par la mémoire (Memory Bandwidth limited)**, et guide ainsi les efforts d'optimisation.

11.7. Conclusion et Pièges à Éviter

Le parallélisme est essentiel pour l'amélioration des performances, mais il présente des défis significatifs, notamment dans le développement de logiciels parallèles efficaces. La tendance actuelle est à l'hybridation, avec des systèmes MIMD dont chaque cœur est doté de puissantes unités SIMD.

Erreurs et Pièges Courants

- **Ignorer la Loi d'Amdahl** : Croire que l'on peut obtenir une accélération linéaire simplement en ajoutant plus de cœurs, sans tenir compte de la partie séquentielle du code (s'applique aux cas de Strong Scaling).
- **Développement Logiciel Inadapté** : Ne pas concevoir le logiciel pour tirer parti d'une architecture multiprocesseur, par exemple en utilisant un seul verrou pour une ressource partagée, ce qui sérialise les accès.
- **Négliger la Bande Passante Mémoire** : Se concentrer sur l'amélioration des performances de calcul vectoriel sans améliorer l'accès à la mémoire. La performance restera bloquée par le toit incliné du diagramme Roofline.
- **Ignorer les Coûts de Démarrage** : Pour les architectures vectorielles, des vecteurs longs sont nécessaires pour amortir les frais généraux et obtenir une réelle accélération.

12.Efficacité Énergétique et Durabilité dans l'Informatique

12.1.Résumé Exécutif

L'analyse de l'efficacité et de la durabilité des systèmes informatiques révèle un lien direct et puissant entre l'optimisation des logiciels, la consommation énergétique et l'impact environnemental global. L'optimisation des performances, notamment par le parallélisme (SIMD, MIMD), n'est pas seulement un objectif technique mais une nécessité écologique. Une étude de cas démontre qu'un service optimisé peut réduire son empreinte carbone par un facteur de sept par rapport à une version non optimisée, principalement en diminuant drastiquement le nombre de serveurs requis.

Cette réduction de matériel met en évidence l'importance critique des émissions du Scope 3 (fabrication), qui peuvent largement dépasser celles du Scope 2 (consommation électrique) pour des services bien conçus. L'utilisation d'accélérateurs comme les GPU est très bénéfique pour l'efficacité énergétique des charges de travail volumineuses et hautement parallèles, bien qu'ils soient contre-productifs pour des tâches plus petites en raison de leur surcoût énergétique et de leur temps de démarrage.

Enfin, une évaluation complète de l'impact environnemental doit dépasser la simple mesure des émissions de carbone pour adopter une approche holistique comme l'Analyse du Cycle de Vie (ACV) et prendre en compte le cadre plus large des limites planétaires, afin d'éviter une "vision tunnel du carbone" et d'aborder la durabilité de manière systémique.

12.2.Analyse de la Performance par le Parallélisme

L'efficacité du parallélisme est démontrée à travers un projet de référence centré sur la multiplication de matrices et la recherche de similarités de motifs. L'analyse des performances de ce service permet d'identifier les facteurs les plus influents et leurs interactions.

Importance et Tendance des Paramètres de Performance

Une exploration des paramètres a permis de quantifier leur importance relative sur la performance globale. La taille de la matrice (MATSIZE) est de loin le facteur le plus déterminant.

Caractéristique	Importance
MATSIZE	0.7522
NBPATTERNS	0.1958
WRK_THREADS	0.0208
WRK_CONN	0.0186
PATTERNSIZE	0.0126

Les données de tendance, bien qu'informatives, sont présentées sans écart-type, ce qui constitue une limite à leur interprétation. Il est également noté que l'impossibilité d'exécuter des scénarios

où `PATTERNSIZE` est supérieur à `MATSIZE` explique pourquoi `NBPATTERNS` semble n'avoir aucun effet apparent.

Interactions entre les Paramètres

Une analyse de la variance (ANOVA) a été utilisée pour étudier les interactions entre les paramètres clés. Les résultats (faibles p-values) indiquent des interactions probables, notamment entre `MATSIZE` et les autres variables. De manière surprenante, aucune interaction significative n'est détectée entre la taille des motifs (`PATTERNSIZE`) et leur nombre (`NBPATTERNS`).

ANOVA (p-value)	NBPATTERNS	PATTERNSIZE	REQUESTS_PER_SECOND
MATSIZE	0.004182	0.002489	0.000018
NBPATTERNS		0.6406	0.011155
PATTERNSIZE			0.020415

Impact de la Taille de la Matrice et des Motifs

L'analyse visuelle des données confirme les observations quantitatives :

- **Taille de la matrice :** La performance, mesurée en requêtes par seconde (RPS), chute de manière drastique à mesure que la taille de la matrice augmente. Pour les petits motifs, elle passe de près de 100 000 RPS pour les plus petites matrices à moins de 1 000 RPS pour des matrices de taille moyenne.
- **Taille et nombre de motifs :** Une carte thermique montre que l'effet de la taille des motifs est similaire à celui de leur nombre. Cela permet de simplifier l'analyse en deux cas principaux : "petits motifs" et "grands motifs". Le point de performance optimal se situe pour un petit nombre de motifs de petite taille.

Analyse de la Latence et du Débit (RPS)

L'étude de la latence en fonction du débit d'entrée (Input RPS) révèle un point de saturation critique. Pour une configuration donnée (`MATSIZE=512`, `NBPATTERNS=1`, `PATTERNSIZE=16`) :

- Le débit de sortie (RPS) atteint un plateau autour de 9 RPS.
- La latence moyenne reste faible et stable jusqu'à environ 8 RPS, puis augmente de façon exponentielle.
- **Conclusion clé :** Tenter d'envoyer des requêtes à un rythme supérieur au point de saturation (~10 RPS dans cet exemple) est inutile, car cela n'augmente pas le débit mais dégrade considérablement la latence.

12.3.Efficacité Énergétique

L'efficacité énergétique est la mesure de la performance par unité d'énergie consommée. Elle est fondamentale pour réduire les coûts opérationnels et l'impact environnemental.

Concepts Clés : Puissance et Énergie

- **Puissance (Watt) :** La consommation instantanée d'un appareil.
- **Énergie (Joule) :** La puissance consommée sur une période donnée ($\text{Énergie (J)} = \text{Temps (s)} \times \text{Puissance (W)}$). Un Watt-heure (Wh) équivaut à 3600 Joules.
- **Métrique d'efficacité :** Le nombre de requêtes traitées par Joule (Requêtes/Joule). Un abus de langage courant est d'utiliser les "Requêtes par Watt" (RPS/Watt). Par exemple, une machine consommant 125 Watts pour traiter 25 RPS a une efficacité de 0,2 RPS/Watt.

Optimisation du Parallélisme (SIMD et MIMD)

L'utilisation de techniques de parallélisme est essentielle pour améliorer l'efficacité.

- **SIMD (Single Instruction, Multiple Data) :** L'accélération obtenue avec SIMD est plus élevée pour les grandes matrices, conformément à la loi d'Amdahl.
- **MIMD (Multiple Instruction, Multiple Data) :** L'utilisation de plusieurs cœurs.
- **Combinaison SIMD+MIMD :** Ensemble, ces deux approches peuvent conduire à une efficacité très élevée, atteignant des facteurs d'amélioration de l'ordre de x25.

Compromis entre Fréquence et Nombre de Cœurs (DVFS)

L'ajustement dynamique de la fréquence et de la tension (DVFS) permet de gérer la consommation d'énergie.

- **Arbitrage :** Pour des tâches comme celles d'un pare-feu, il est plus efficace d'utiliser plus de cœurs à une fréquence plus basse plutôt que moins de cœurs à une fréquence élevée.
- **Le "faux gain" du DVFS :** Réduire la fréquence diminue la consommation en Watts, mais augmente le temps d'exécution. Si la tâche prend deux fois plus de temps pour une réduction de 20% des Watts, la consommation totale d'énergie (en Joules) augmente. Il faut donc viser la réduction des Joules, pas seulement des Watts.

12.4.Évaluation de l'Impact Environnemental

L'optimisation des performances a un impact direct et quantifiable sur l'empreinte écologique d'un service.

Le Protocole GHG et l'Empreinte Carbone

Le protocole sur les gaz à effet de serre (GHG Protocol) est la norme pour calculer l'empreinte carbone. Il se divise en trois périmètres (scopes) :

- **Scope 1 :** Émissions directes de l'entreprise (bâtiments, véhicules). Généralement faible pour les entreprises du secteur des TIC.
- **Scope 2 :** Émissions indirectes liées à la consommation d'électricité.
- **Scope 3 :** Toutes les autres émissions indirectes, principalement celles de la chaîne d'approvisionnement (ex: fabrication du matériel).

Étude de Cas : "Bon Élève" vs. "Mauvais Élève"

Ce cas compare l'impact carbone d'un service pour servir 100 000 requêtes/seconde en continu pendant 5 ans, en se basant sur une solution logicielle mal optimisée ("mauvais élève") et une solution bien optimisée ("bon élève").

Hypothèses :

- Performance "mauvais élève" : 150 req/s par machine.
- Performance "bon élève" : 1100 req/s par machine.
- Consommation d'un serveur : 150 Watts.
- Émissions de fabrication (Scope 3) : 2500 kg CO₂e par serveur.
- Intensité carbone de l'électricité (Belgique) : 165 g CO₂e/kWh.

Indicateur	Mauvais Élève	Bon Élève	Amélioration
Machines requises	666	90	7.4x
Émissions Scope 2 (5 ans)	723 t CO ₂ e	98 t CO ₂ e	7.4x
Émissions Scope 3 (5 ans)	1665 t CO ₂ e	225 t CO ₂ e	7.4x
Total CO₂e (5 ans)	2388 t CO₂e	325 t CO₂e	~7.3x

Conclusion : L'optimisation logicielle aboutit à une amélioration de plus de 7 fois l'empreinte carbone du service. Le service du "mauvais élève" émet autant que 40 vols New-York-Paris.

Analyse du Cycle de Vie (ACV)

L'ACV est une méthode plus complète que le protocole GHG, évaluant l'ensemble des impacts environnementaux d'un produit "du berceau à la tombe" (extraction des matières premières, fabrication, transport, utilisation, fin de vie).

- **ACV d'un PC :** L'énergie intrinsèque est répartie de manière quasi égale entre la fabrication (38.1%) et la phase d'utilisation (39.7%).
- **ACV d'un serveur (Dell R740) :** La production des disques SSD (3.84TB) représente la part la plus importante (78.8%) de l'impact sur le réchauffement climatique de la fabrication du serveur.

Au-delà du Carbone : Les Limites Planétaires

Il est crucial de ne pas tomber dans la "vision tunnel du carbone", qui se concentre uniquement sur les émissions de GES. La durabilité doit être abordée dans le cadre des 9 limites planétaires (changement climatique, intégrité de la biosphère, etc.). En 2025, 7 de ces 9 limites ont été franchies, ce qui souligne l'urgence d'une approche systémique.

12.5. Le Rôle des GPU dans l'Amélioration de l'Efficacité

Les GPU, avec leur architecture SIMT (Single Instruction, Multiple Threads), offrent un potentiel d'accélération significatif.

Performance et Consommation Énergétique

- **Performance** : Les GPU sont très efficaces pour les grandes matrices mais inefficaces pour les petites en raison du surcoût lié à leur initialisation. Pour être efficaces, ils doivent être "alimentés" en données suffisamment rapidement, ce qui nécessite plusieurs cœurs CPU. La combinaison MIMD (CPU) + SIMT (GPU) est la plus performante pour les charges de travail importantes.
- **Consommation** : Cette haute performance a un coût énergétique. La puissance consommée par une machine avec GPU en pleine charge peut atteindre plus de 275 Watts, contre environ 150 Watts pour une solution CPU seule.

Efficacité en Joules par Requête

Malgré leur consommation élevée, les GPU peuvent être extrêmement efficaces en termes d'énergie par tâche.

- **Grandes charges de travail (ex: matrice 512x512)** : Une solution MIMD (CPU) + SIMT (GPU) est environ 4 fois plus efficace (moins de Joules/Requête) qu'une solution GPU seule et bien plus efficace que les solutions purement CPU.
- **Petites charges de travail (ex: matrice < 128x128)** : L'utilisation du GPU est contre-productive. Les solutions basées sur le CPU (SIMD, MIMD+SIMD) consomment moins d'énergie par requête.

Impact Carbone (Scope 3)

L'ajout d'un GPU à un serveur a un impact sur les émissions de fabrication (Scope 3).

- L'empreinte carbone de fabrication d'un GPU est estimée à environ 100 kg de CO₂e.
- Comparé aux ~2500 kg de CO₂e pour un serveur, cet ajout est largement justifié si la charge de travail est adaptée, car les gains en performance et en efficacité permettent de réduire considérablement le nombre total de serveurs nécessaires.

12.6.Conclusion

Cette analyse met en lumière plusieurs points fondamentaux pour la conception de services informatiques durables :

1. **L'impact de la conception** : Les choix de conception logicielle ont un impact direct et massif sur les besoins matériels et, par conséquent, sur l'empreinte carbone totale d'un service.
2. **L'efficacité comme levier** : L'optimisation des performances n'est pas une fin en soi, mais un moyen essentiel pour réduire la consommation d'énergie et les émissions de GES.
3. **Le bon outil pour la bonne tâche** : Des technologies comme les GPU sont de puissants outils d'efficacité, mais leur utilisation doit être limitée aux charges de travail pour lesquelles ils sont conçus (larges et parallèles).
4. **Une vision holistique** : Une évaluation environnementale complète doit aller au-delà du carbone en utilisant des cadres comme l'ACV et en tenant compte des limites planétaires pour une véritable transition vers la durabilité.

13. Entrées/Sorties Matérielles

13.1. Résumé Exécutif

Ce document synthétise les principes fondamentaux des entrées/sorties (E/S) matérielles, de l'architecture des systèmes modernes à l'analyse de la performance. L'analyse révèle que l'efficacité des processeurs graphiques (GPU) dépend fortement de la charge de travail ; ils excellent pour les tâches massivement parallèles (grandes matrices) où ils peuvent être jusqu'à quatre fois plus efficaces énergétiquement que les solutions CPU, mais s'avèrent inefficaces pour les petites tâches en raison des frais de démarrage. L'architecture matérielle a évolué de bus partagés vers des interconnexions dédiées à haute vitesse comme le PCIe et l'Intel UPI, avec une intégration croissante des contrôleurs au sein du CPU. La communication avec les périphériques s'effectue via deux méthodes principales : les E/S mappées par port (PMIO), une méthode lente et synchrone, et les E/S mappées en mémoire (MMIO), plus performantes et flexibles. Pour les transferts de données volumineux, l'Accès Direct à la Mémoire (DMA) est essentiel, car il permet de décharger le CPU en gérant les transferts de manière autonome. Enfin, les interruptions constituent le mécanisme principal de communication asynchrone entre les périphériques et le CPU, remplaçant le *polling* inefficace et gérant des centaines de milliers d'événements par seconde sur les serveurs modernes.

13.2. Analyse de la Performance et de l'Efficacité du GPU

L'évaluation de la performance du GPU, notamment dans le contexte de calculs matriciels, démontre un compromis clair entre la taille de la charge de travail, la puissance de calcul, et l'efficacité énergétique.

Performance en Termes de Requêtes par Seconde

Les données de performance montrent que l'approche SIMT (Single Instruction, Multiple Threads) utilisée par les GPU est inefficace pour les petites matrices. Cela est dû aux frais généraux (*overhead*) importants liés au démarrage des calculs. Le GPU nécessite que ses multiples cœurs soient alimentés en données rapidement pour atteindre son plein potentiel, ce qui n'est pas réalisable avec de petites charges de travail. En revanche, pour les matrices de grande taille, les approches combinant CPU et GPU (MIMD+SIMT) offrent des performances élevées, bien que le débit diminue à mesure que la complexité des calculs augmente.

Consommation Électrique et Efficacité Énergétique

La haute performance a un coût énergétique significatif. Les configurations utilisant le GPU, seul ou avec le CPU, affichent la consommation électrique la plus élevée, atteignant près de 300 Watts pour certaines tailles de matrice. Cependant, en analysant l'efficacité en termes de Joules par requête, le GPU se révèle très avantageux pour les charges de travail importantes.

- **Pour les petites charges de travail :** L'utilisation d'un GPU n'est pas recommandée si la charge n'est pas massivement parallèle. Son coût énergétique par requête est plus élevé que celui des approches CPU optimisées (MIMD ou MIMD+SIMD).

- **Pour les grandes charges de travail** : Le GPU devient extrêmement efficace. Pour une matrice de taille 512, il est **quatre fois plus efficace** que l'approche SIMD sur CPU. La combinaison CPU+GPU (MIMD+SIMT) est la plus efficace de toutes.

Impact Carbone

L'empreinte carbone d'un GPU est un facteur à considérer. Un GPU a un carbone intrinsèque ("embodied carbon") estimé à environ **100 kg de CO₂e**. Comparé à l'empreinte carbone d'un serveur (estimée à 2500 kg de CO₂e), l'ajout d'un GPU reste justifié au vu des gains d'efficacité qu'il procure sur les charges de travail adaptées.

13.3.Architecture et Évolution du Matériel d'E/S

La gestion des E/S est une composante majeure de la conception d'un système d'exploitation. L'architecture matérielle qui la sous-tend a considérablement évolué.

Évolution des Structures de Bus

- **Anciennes Architectures** : Les anciens PC utilisaient une structure de bus unique (par ex. PCI), où deux périphériques ne pouvaient pas communiquer simultanément. Le contrôleur mémoire et le cache étaient externes au processeur.
- **Architectures Modernes** : Les systèmes actuels utilisent des interconnexions point à point. Le bus PCIe est désormais composé de "lignes" (*lanes*) dédiées, permettant à des groupes de lignes d'être utilisés séparément. Le contrôleur mémoire et le cache sont intégrés directement dans le CPU. Pour les systèmes multi-CPU, un bus dédié comme l'Intel UPI (Ultra Path Interconnect) assure la communication inter-processeur.

Le Standard PCIe et ses Débits

Le bus PCI Express (PCIe) est devenu l'interface standard pour les périphériques à haute vitesse. Sa vitesse a évolué au fil des générations :

Version PCIe	Transferts par Seconde (par ligne)	Débit par Ligne (Encodage)	Débit x16
PCIe 1.0	2.5 GT/s	0.25 GB/s (8b/10b)	4 GB/s
PCIe 2.0	5.0 GT/s	~0.5 GB/s (8b/10b)	8 GB/s
PCIe 3.0	8.0 GT/s	0.985 GB/s (128b/130b)	~16 GB/s
PCIe 4.0	16.0 GT/s	~1.97 GB/s (128b/130b)	~32 GB/s
PCIe 5.0	32.0 GT/s	~3.94 GB/s (128b/130b)	~64 GB/s

Exemple Concret : CPU Intel i7-12700F et Chipset Z690

- **CPU (i7-12700F)** : Ce processeur offre **20 lignes PCIe** (supportant les versions 5.0 et 4.0) et **8 lignes DMI 4.0** (Direct Media Interface) pour la communication avec le chipset. Il dispose également de deux canaux mémoire, ce qui rend une configuration de 2x16 Go de RAM plus performante que 1x32 Go.

- **Chipset (Z690)** : Le chipset agit comme un concentrateur, "étalant" les lignes DMI provenant du CPU en de multiples ports d'E/S : lignes PCIe supplémentaires (3.0 et 4.0), ports USB, ports SATA, et contrôleurs réseau (Wi-Fi, Ethernet).

Allocation des Lignes PCIe sur une Carte Mère

L'exemple de la carte mère ASRock Z690 Taichi montre comment les lignes PCIe sont distribuées :

- **Slots connectés au CPU** : Deux slots PCIe 5.0 x16 sont directement reliés au CPU. Cependant, ils se partagent les lignes : si les deux sont utilisés, ils fonctionnent en mode x8/x8.
- **Slots connectés au Chipset** : Un slot PCIe 4.0 x16 et un slot PCIe 3.0 x1 sont gérés par le chipset. Il est crucial de noter que le slot x16 du chipset n'est en réalité câblé qu'en x4, limitant son débit maximal.

Pour des performances optimales, un GPU doit être branché sur un slot PCIe directement connecté au CPU.

13.4.Mécanismes de Communication avec les Périphériques

Les pilotes de périphériques (*device drivers*) encapsulent les détails matériels et présentent une interface uniforme au système d'exploitation. La communication repose sur des contrôleurs et des registres.

Contrôleurs et Registres

Chaque périphérique est géré par un **contrôleur** (ou adaptateur hôte), une puce électronique contenant un processeur, de la mémoire privée et de la micro-logique. Le pilote communique avec le contrôleur via des **registres** spécifiques :

- **Data-in register** : Pour lire les données du périphérique.
- **Data-out register** : Pour écrire des données vers le périphérique.
- **Status register** : Contient des bits indiquant l'état du périphérique (ex: occupé, erreur).
- **Control register** : Pour envoyer des commandes au périphérique.

E/S Mappées par Port (Port-Mapped I/O - PMIO)

Le PMIO utilise un espace d'adressage distinct de la mémoire principale, accessible via des instructions CPU dédiées (par ex. `IN` et `OUT` sur x86).

- **Inconvénients** : Cette méthode est **extrêmement lente** car elle est synchrone (le CPU attend la fin de l'opération) et les transferts se font octet par octet.
- **Étude de cas (Port parallèle)** : Écrire à l'adresse de port `0x378` envoie directement des données sur les broches du port parallèle. L'appel système `outb()` en C est un wrapper pour l'instruction assembleur `OUT`.

E/S Mappées en Mémoire (Memory-Mapped I/O - MMIO)

Le MMIO intègre les registres de commande et de données des périphériques dans l'espace d'adressage de la mémoire physique.

- **Avantages** : Les accès se font via des instructions mémoire standard (`mov`, `add`, etc.), ce qui est beaucoup plus rapide et simple à programmer.
- **Fonctionnement** : Le BIOS énumère les périphériques au démarrage, leur demande la taille de mémoire dont ils ont besoin, et alloue des plages d'adresses correspondantes. Le système d'exploitation peut ensuite ré-allouer ces plages, notamment pour les périphériques gourmands en mémoire comme les GPU.

13.5.Méthodes d'Accès aux Données et de Transfert

Une fois la communication établie, le transfert de données peut être géré de différentes manières.

Le Polling (Sondage)

Le *polling* est une technique de "busy-waiting" où le CPU vérifie de manière répétée le bit d'état d'un périphérique dans une boucle pour savoir s'il est prêt. Cette méthode est simple mais **très inefficace** car elle monopolise le CPU, surtout si le périphérique est lent. Elle est acceptable uniquement pour les périphériques très rapides.

Les Interruptions

Le mécanisme d'interruption est une approche asynchrone et efficace :

1. Le pilote de périphérique lance une opération d'E/S.
2. Le contrôleur du périphérique exécute l'opération.
3. Une fois l'opération terminée, le contrôleur envoie un signal d'interruption au CPU via une ligne dédiée (*interrupt-request line*).
4. Le CPU arrête sa tâche en cours, sauvegarde son contexte, et exécute un gestionnaire d'interruption (*interrupt handler*) spécifique à l'événement.
5. Le gestionnaire traite les données, puis le CPU reprend sa tâche initiale.

Les interruptions sont omniprésentes dans les systèmes modernes. Un ordinateur de bureau au repos peut générer des dizaines de milliers d'interruptions en quelques secondes. Elles sont également utilisées pour gérer les exceptions, les erreurs mémoire (*page faults*) et les appels système.

Accès Direct à la Mémoire (Direct Memory Access - DMA)

Pour transférer de gros volumes de données (par ex., 1 Go vers la mémoire d'un GPU), utiliser le MMIO instruction par instruction serait trop lent (ex: 256 millions d'instructions `mov`). Le DMA résout ce problème.

- **Principe** : Le DMA est un moteur spécialisé, souvent intégré au CPU, qui gère les transferts de données entre un périphérique et la mémoire RAM en **contournant le CPU**.
- **Fonctionnement** :

1. L'OS écrit un "bloc de commande DMA" en mémoire, spécifiant les adresses source et destination, le mode (lecture/écriture) et la quantité de données.
2. L'OS transmet l'adresse de ce bloc au contrôleur DMA.
3. Le contrôleur DMA effectue le transfert de manière autonome. Pendant ce temps, le **CPU est libre** pour exécuter d'autres tâches.
4. Une fois le transfert terminé, le contrôleur DMA envoie une interruption au CPU.

Le DMA est une forme de **déchargement** (*offloading*), essentielle pour les périphériques à haut débit comme les cartes réseau, les contrôleurs de stockage et les GPU.

14.Sous-système d'Entrée/Sortie du Noyau et la Performance

14.1.Résumé Exécutif

Le sous-système d'Entrée/Sortie (E/S) du noyau est une composante fondamentale des systèmes d'exploitation modernes, agissant comme une couche d'abstraction essentielle entre les processus de l'espace utilisateur et la complexité du matériel. Sa fonction principale est de gérer la communication avec une vaste gamme de périphériques, en masquant leurs spécificités matérielles derrière une interface unifiée, telle que le Virtual File System (VFS). Cette uniformisation est rendue possible par les pilotes de périphériques, qui traduisent les requêtes génériques en commandes spécifiques aux contrôleurs matériels.

La performance des opérations d'E/S est un enjeu majeur, influencé par plusieurs facteurs et optimisé par diverses techniques. Le système d'exploitation emploie des modèles d'opérations distincts — bloquantes, non bloquantes et asynchrones — pour répondre aux différents besoins des applications. Des mécanismes comme la mise en tampon (buffering), le caching et le spooling sont cruciaux pour gérer les asynchronies de vitesse et de taille de transfert entre le CPU, la mémoire et les périphériques. De plus, l'ordonnancement intelligent des requêtes, en particulier pour les disques durs (par ex., l'algorithme SCAN), minimise les latences mécaniques et améliore le débit global.

Pour la gestion du stockage, la technologie RAID (Redundant Array of Individual Disks) permet de combiner plusieurs disques pour augmenter la vitesse (RAID 0), assurer la redondance des données (RAID 1), ou offrir un compromis équilibré entre les deux (RAID 5, RAID 10). Les défis de performance, tels que les changements de contexte, la copie de données et la charge CPU, sont des goulots d'étranglement reconnus. Les approches modernes pour surmonter ces obstacles incluent l'utilisation du DMA (Direct Memory Access) et des techniques plus avancées comme le RDMA (Remote Direct Memory Access), qui visent à contourner le noyau pour permettre des communications à très haute performance, principalement dans les centres de données. La gestion de l'alimentation est également une responsabilité clé du sous-système, particulièrement dans les environnements mobiles où l'efficacité énergétique est primordiale.

14.2.Architecture et Interface du Sous-système d'E/S

Le sous-système d'E/S du noyau constitue une couche logicielle intermédiaire entre les applications et le matériel physique. Les processus n'interagissent pas directement avec les périphériques ; ils le font via des **appels système** qui déclenchent une transition de l'espace utilisateur vers l'espace noyau (trap to kernel).

L'architecture est structurée en plusieurs couches :

1. **Matériel (Hardware) :** Les périphériques physiques (disques, claviers, cartes réseau, etc.).
2. **Contrôleurs de Périphériques (Device Controllers) :** Composants électroniques qui opèrent les périphériques.

3. **Pilotes de Périphériques (Device Drivers)** : Logiciels spécifiques à chaque type de contrôleur, qui masquent les détails d'implémentation au reste du noyau.
4. **Sous-système d'E/S du Noyau (Kernel I/O Subsystem)** : Fournit des services généraux comme le buffering, le caching, le spooling, la planification et la protection.
5. **Noyau (Kernel)** : Intègre le sous-système d'E/S et expose les appels système aux applications.

Le **Virtual File System (VFS)** est une couche d'indirection cruciale qui offre une interface de fichiers uniforme pour différents systèmes de fichiers et types de périphériques, permettant aux applications d'utiliser des appels comme `open()`, `read()`, et `write()` de manière générique.

14.3. Caractéristiques et Classification des Périphériques d'E/S

Les périphériques d'E/S varient considérablement et peuvent être classifiés selon plusieurs dimensions, ce qui influence la manière dont le noyau les gère.

Aspect	Variation	Exemples
Mode de transfert de données	Caractère, Bloc	Terminal, Disque
Méthode d'accès	Séquentiel, Aléatoire	Modem, CD-ROM
Planification de transfert	Synchrone, Asynchrone	Bande, Clavier
Partage	Dédié, Partageable	Bande, Clavier
Vitesse du périphérique	Latence, Temps de recherche (seek time), Taux de transfert, Délai inter-opérations	
Direction des E/S	Lecture seule, Écriture seule, Lecture-écriture	CD-ROM, Contrôleur graphique, Disque

14.4. Modèles d'Opérations d'E/S

Le noyau propose plusieurs modèles pour gérer le flux de contrôle d'un processus lors d'une opération d'E/S.

- **E/S Bloquante (Blocking I/O)** : Le processus est suspendu jusqu'à ce que l'opération d'E/S soit terminée. C'est le modèle le plus simple à utiliser, mais il peut être inefficace si le processus a d'autres tâches à accomplir.
- **E/S Non Bloquante (Non-blocking I/O)** : L'appel système retourne immédiatement, avec autant de données qu'il peut transférer sans attendre. Si aucune donnée n'est prête, il retourne une indication d'échec ou un compte de zéro octet. Le processus doit souvent réitérer l'appel (polling). L'appel `select()` peut être utilisé pour vérifier si des données sont prêtes avant de tenter une lecture ou une écriture.
- **E/S Asynchrone (Asynchronous I/O)** : Le processus initie l'opération d'E/S et continue son exécution immédiatement. Le sous-système d'E/S notifie le processus (via un signal ou une fonction de rappel) lorsque l'opération est terminée. Ce modèle est puissant mais plus complexe à mettre en œuvre pour les développeurs.

14.5.Mécanismes d'Optimisation des E/S

Pour améliorer la performance, le sous-système d'E/S utilise plusieurs techniques.

Mise en Tampon (Buffering)

Un tampon (buffer) est une zone de mémoire utilisée pour stocker temporairement des données lors de leur transfert. Ses objectifs sont :

- **Gérer les discordances de vitesse** : Le CPU et la mémoire sont beaucoup plus rapides que la plupart des périphériques d'E/S. Le tampon permet au CPU de déposer des données et de passer à autre chose sans attendre le périphérique.
- **Adapter les tailles de transfert** : Les périphériques peuvent avoir des contraintes sur la taille des blocs de données (par ex., secteurs de disque). Le tampon accumule les petites écritures pour former un bloc de taille adéquate.
- **Sémantique de copie** : Pour éviter que les données dans l'espace utilisateur ne soient modifiées après un appel système mais avant que le transfert physique n'ait eu lieu, les données sont copiées dans un tampon du noyau.

Types de tampons :

- **Double Tamponnage** : Utilise deux tampons. Pendant que le périphérique traite les données d'un tampon, le noyau peut remplir le second, et vice versa.
- **Tampon Circulaire (Ring Buffer)** : Une structure de données efficace avec des pointeurs de tête et de queue, utile lorsque le producteur et le consommateur de données ont des vitesses très différentes (par ex., communication réseau).

Ordonnancement des Requêtes

Le noyau peut réordonnancer les requêtes d'E/S dans la file d'attente d'un périphérique pour optimiser la performance globale, particulièrement pour les disques durs magnétiques où le déplacement de la tête de lecture/écriture (seek) est coûteux.

- **FCFS (First-Come, First-Served)** : Traite les requêtes dans leur ordre d'arrivée. Simple mais potentiellement inefficace, car il peut entraîner des mouvements de tête erratiques.
- **SCAN (Algorithme de l'ascenseur)** : La tête du disque se déplace dans une seule direction (par ex., de l'intérieur vers l'extérieur), traitant toutes les requêtes sur son chemin. Une fois l'extrémité atteinte, elle inverse sa direction. Cela réduit considérablement le temps de recherche moyen, mais peut être inéquitable pour les requêtes situées aux extrémités du disque.

E/S Vectorielles (Vectored I/O)

Également connu sous le nom de "**scatter-gather**", ce mécanisme permet à une seule opération d'E/S (un seul appel système, comme `readve()` sous Unix) de lire des données depuis une source vers plusieurs tampons en mémoire, ou d'écrire des données depuis plusieurs tampons vers une destination. Cela réduit le surcoût lié aux appels système et aux changements de contexte par rapport à de multiples appels individuels.

14.6. Technologie RAID (Redundant Array of Individual Disks)

Le RAID est une technologie permettant de combiner plusieurs disques durs physiques en une seule unité logique pour améliorer la performance, la redondance ou les deux.

- **RAID 0 (Agrégation par bandes / Striping)** : Les données sont réparties en blocs sur plusieurs disques. Offre une bande passante doublée (pour deux disques) mais aucune redondance. La défaillance d'un seul disque entraîne la perte de toutes les données.
- **RAID 1 (Mise en miroir / Mirroring)** : Les données sont dupliquées sur deux disques ou plus. Offre une excellente redondance : si un disque tombe en panne, les données sont toujours disponibles sur l'autre.
- **RAID 4 (Parité sur disque dédié)** : Utilise un disque pour stocker les informations de parité (calculées par une opération XOR sur les blocs de données des autres disques). Permet de reconstruire les données d'un disque défaillant. Le disque de parité devient un goulot d'étranglement lors de nombreuses petites opérations d'écriture.
- **RAID 5 (Parité répartie)** : Similaire au RAID 4, mais les informations de parité sont réparties sur tous les disques de la grappe, éliminant ainsi le goulot d'étranglement du disque de parité.
- **RAID 6 (Double parité)** : Fournit une redondance supplémentaire en calculant deux ensembles d'informations de parité (par ex., via des codes de Reed-Solomon), permettant à la grappe de survivre à la défaillance de deux disques.
- **RAID 10 (ou 1+0)** : Combine le striping et le mirroring. C'est une grappe de disques en miroir qui est ensuite agrégée par bandes. Offre à la fois de hautes performances et une bonne redondance.

14.7. Autres Services du Sous-système d'E/S

Le sous-système d'E/S fournit un ensemble de services coordonnés pour les applications et le reste du noyau :

- **Mise en Cache (Caching)** : Une copie des données est conservée sur un périphérique plus rapide (la mémoire principale) pour accélérer les accès futurs. Dans de nombreux OS, les pages de fichiers lues sont conservées en mémoire jusqu'à ce que celle-ci soit requise pour autre chose.
- **Spooling** : Pour les périphériques ne pouvant traiter qu'une seule requête à la fois (comme une imprimante), les sorties sont mises en file d'attente sur un disque.
- **Gestion des Erreurs** : Le système d'exploitation peut tenter de récupérer des erreurs (par ex., réessayer une lecture de disque) et consigne les problèmes dans des journaux système (`dmesg` sous Linux).
- **Protection des E/S** : Pour des raisons de sécurité et de stabilité, les instructions d'E/S sont privilégiées. Les processus utilisateur doivent passer par des appels système pour effectuer des opérations d'E/S, permettant au noyau de valider et de contrôler les accès.

14.8. Gestion de l'Alimentation

La gestion de l'alimentation est une fonction essentielle de l'OS, étroitement liée aux E/S.

- **Objectif** : Réduire la consommation d'énergie et la production de chaleur en mettant hors tension les composants inutilisés.
- **Approche Android** : Gère l'alimentation au niveau des composants en construisant un arbre représentant la topologie des périphériques. Une branche entière de l'arbre peut être désactivée si tous ses composants sont inutilisés. Android utilise également des `wake locks` pour empêcher le système de se mettre en veille lorsqu'une tâche critique est en cours, et le `power collapse` pour un mode de veille très profond.
- **ACPI (Advanced Configuration and Power Interface)** : Une norme industrielle qui permet au système d'exploitation de gérer la configuration et l'alimentation des périphériques, plutôt que de s'en remettre au BIOS.

14.9. Analyse et Amélioration de la Performance des E/S

Le cycle de vie d'une requête d'E/S, de l'appel système initial au retour au processus, implique de nombreuses étapes qui peuvent devenir des goulots d'étranglement.

Goulots d'étranglement courants :

- **Charge CPU** : L'exécution des pilotes de périphériques et du code du noyau consomme des cycles CPU.
- **Changements de contexte** : Les interruptions des périphériques et les appels système forcent des changements de contexte, qui ont un coût en performance.
- **Copie de données** : Le transfert de données entre les tampons de l'espace noyau et ceux de l'espace utilisateur est une source de surcoût.

Stratégies d'amélioration :

- **Réduire les changements de contexte et les copies de données.**
- **Utiliser le DMA (Direct Memory Access)** : Permet à un périphérique de transférer des données directement vers/depuis la mémoire principale sans impliquer le CPU.
- **Réduire les interruptions** : En utilisant des transferts de grande taille et des contrôleurs matériels intelligents.
- **Évitement du Noyau (Kernel Bypass)** : Pour les applications nécessitant une latence minimale, les pilotes peuvent être implémentés en espace utilisateur, utilisant le MMIO (Memory-mapped I/O) et le DMA pour un accès direct au matériel. Cela élimine les changements de contexte mais sacrifie les services du noyau (pile TCP, sécurité, etc.).
- **RDMA (Remote Direct Memory Access)** : Une technologie avancée, principalement utilisée dans les centres de données, qui permet à un serveur d'écrire directement dans la mémoire d'un autre serveur sur le réseau, en contournant les noyaux et les CPU des deux machines. Cela réduit considérablement la latence de communication.

15.Mémoire Virtuelle

15.1.Résumé Exécutif

La mémoire virtuelle est un mécanisme fondamental des systèmes d'exploitation modernes qui crée une abstraction de la mémoire physique (RAM) pour chaque processus. Gérée par l'Unité de Gestion de la Mémoire (MMU), une composante matérielle intégrée au CPU, elle traduit les adresses virtuelles générées par un programme en adresses physiques réelles. Cette abstraction offre des avantages cruciaux :

1. **Isolation des Processus** : Chaque processus dispose de son propre espace d'adressage virtuel, l'empêchant d'accéder ou de modifier la mémoire d'autres processus ou du système d'exploitation, garantissant ainsi la stabilité et la sécurité du système.
2. **Gestion Efficace de la Mémoire** : Elle atténue la fragmentation de la mémoire physique en permettant à un programme de voir un bloc de mémoire contigu virtuellement, même si les pages physiques correspondantes sont dispersées.
3. **Optimisation des Ressources** : Des techniques comme l'allocation paresseuse (lazy allocation) et la pagination (swapping) sur disque permettent d'exécuter des programmes nécessitant plus de mémoire que la RAM disponible, en ne chargeant que les parties activement utilisées et en déplaçant les autres sur un support de stockage secondaire.

Les principaux défis techniques de la mémoire virtuelle, tels que la latence de la traduction d'adresse et la taille des tables de pages, sont surmontés par des solutions matérielles sophistiquées. Le **Translation Lookaside Buffer (TLB)**, un cache dédié aux traductions, accélère considérablement les accès mémoire. Les **tables de pages à plusieurs niveaux** réduisent l'empreinte mémoire nécessaire pour stocker les mappages d'adresses. Enfin, la sécurité est assurée par des modes d'exécution distincts du CPU (privilegié et utilisateur), protégeant les structures de données critiques du système d'exploitation contre les modifications non autorisées par les applications.

15.2.Introduction aux Concepts Fondamentaux

Le Principe de la Mémoire Virtuelle

La mémoire virtuelle offre à chaque processus une vue unique et privée de la mémoire. Alors que plusieurs programmes peuvent s'exécuter simultanément et utiliser des adresses identiques (par exemple, `0x2000`), la mémoire virtuelle garantit que ces adresses pointent vers des emplacements distincts dans la mémoire physique. La mémoire est donc "virtuelle" car les adresses utilisées par un programme ne correspondent pas directement aux adresses physiques de la RAM.

L'Unité de Gestion de la Mémoire (MMU)

La traduction entre les adresses virtuelles et physiques est effectuée par une composante matérielle appelée l'Unité de Gestion de la Mémoire (MMU). Depuis plusieurs décennies, la MMU est intégrée directement dans le CPU. Le processus est le suivant :

1. Le CPU génère une adresse virtuelle pour un accès mémoire (par exemple, `load 0x1004, r0`).
2. La MMU intercepte cette adresse virtuelle.
3. Elle la traduit en une adresse physique correspondante (par exemple, `0x22004`).
4. Cette adresse physique est ensuite utilisée pour accéder à la mémoire principale (RAM).

15.3.Mécanismes de la Mémoire Paginée

Traduction d'Adresses et Pages

La traduction d'adresses s'effectue par blocs, appelés **pages**. Une adresse virtuelle est divisée en deux parties :

- **Le Numéro de Page Virtuelle (#page)** : Les bits de poids fort de l'adresse, qui identifient la page.
- **Le Décalage (Offset)** : Les bits de poids faible, qui indiquent la position à l'intérieur de cette page.

La MMU utilise une structure de données, la **Table des Pages (Page Table)**, pour mapper un numéro de page virtuelle à un **numéro de trame physique (#frame)**. Le décalage reste inchangé lors de la traduction.

- **Exemple (adresse 32 bits, page de 4 Ko) :**
 - Taille de page de 4 Ko (4096 octets) = 2^{12} octets. Le décalage est donc sur 12 bits.
 - Adresse virtuelle : `0x12413678`.
 - Numéro de page virtuelle (#page) : `0x12413` (les 20 bits de poids fort).
 - Décalage : `0x678` (les 12 bits de poids faible).
 - Si la table des pages mappe `0x12413` à la trame `0x421`, l'adresse physique devient `0x421678`.

Les tailles de page peuvent varier. Les CPU Intel récents supportent des pages de 4 Ko, 2 Mo et 1 Go.

La Table des Pages et ses Indicateurs (Flags)

La table des pages, stockée elle-même en mémoire principale, contient non seulement le mappage `#page` → `#frame` mais aussi des indicateurs (flags) pour chaque entrée.

- **Indicateur de Présence (P)** : Si $P=1$, l'entrée est valide et la page est en RAM. Si $P=0$, l'entrée est invalide.
- **Défaut de Page (Page Fault)** : Tenter d'accéder à une adresse dont l'entrée de table de pages a $P=0$ déclenche une exception matérielle appelée "défaut de page". Le contrôle est alors transféré au système d'exploitation, qui peut soit terminer le programme (erreur critique), soit allouer la mémoire manquante (allocation paresseuse) et reprendre l'exécution.

Indicateur	Description
P (Present)	Indique si la page est présente en mémoire physique.

R/W (Read/Write)	Définit si la page est en lecture seule ($R/W=0$) ou en lecture/écriture ($R/W=1$).
U/S (User/Supervisor)	Définit si la page est accessible en mode utilisateur ($U=1$) ou uniquement en mode privilégié ($U=0$).
A (Accessed)	Mis à 1 par le matériel lorsque la page est accédée (lue ou écrite). Utilisé par l'OS pour les algorithmes de remplacement.
D (Dirty)	Mis à 1 par le matériel lorsque la page est modifiée (écrite). Indique si la page doit être sauvegardée sur disque avant d'être remplacée.

15.4.Utilité et Avantages de la Mémoire Virtuelle

Isolation des Processus

Grâce aux tables de pages, un processus ne peut accéder qu'aux trames physiques listées dans sa propre table. Sous Linux ou Windows, chaque processus se voit attribuer une table de pages distincte par le système d'exploitation. Ainsi, même si deux processus utilisent la même adresse virtuelle, ils accèdent à des emplacements physiques différents, garantissant une isolation complète.

Partage de Mémoire

Le partage de données entre processus est implémenté en mappant la même trame physique ($\#Frame$) dans les tables de pages des deux processus. L'indicateur R/W peut être utilisé pour qu'un processus ait un accès en lecture seule tandis que l'autre peut écrire, permettant un contrôle fin sur les permissions.

Atténuation de la Fragmentation de la Mémoire

Un processus peut demander un grand bloc de mémoire contiguë (par exemple, 12 Ko). La mémoire virtuelle lui donne l'illusion de cet espace contigu en allouant trois pages de 4 Ko qui peuvent être physiquement dispersées dans la RAM. Cela résout le problème de la fragmentation externe, où il y a assez de mémoire libre totale mais pas de bloc contigu suffisamment grand.

Allocation Automatique et Paresseuse (Lazy Allocation)

Le mécanisme de défaut de page est utilisé pour des optimisations comme la **croissance automatique de la pile**. Initialement, l'OS alloue une seule page pour la pile et marque la page adjacente comme non présente ($P=0$). Si le programme a besoin de plus d'espace sur la pile et accède à cette page invalide, un défaut de page se produit. L'OS intercepte l'exception, alloue une nouvelle trame physique, met à jour la table des pages, et relance l'instruction qui a causé le défaut. Le processus est entièrement transparent pour le programme.

La Pagination (Swapping)

La pagination permet à un système d'allouer plus de mémoire que la RAM physiquement disponible en utilisant un espace de stockage secondaire (disque dur ou SSD) comme extension de la mémoire.

- **Swap Out** : Lorsque la mémoire physique est pleine, l'OS sélectionne une page (idéalement peu utilisée), copie son contenu sur le disque, et marque son entrée dans la table des pages comme invalide ($P=0$). La trame physique est alors libérée.
- **Swap In** : Si un programme tente d'accéder à une page qui a été "swappée", un défaut de page se produit. L'OS lit la page depuis le disque, la charge dans une trame physique libre (en effectuant un autre "swap out" si nécessaire), met à jour la table des pages avec $P=1$, et reprend l'exécution du programme.

Pour optimiser ce processus, les indicateurs **Accessed (A)** et **Dirty (D)** sont utilisés. L'indicateur **A** aide les algorithmes de remplacement de pages (comme des approximations de LRU) à choisir les pages les moins récemment utilisées. L'indicateur **D** permet d'éviter une écriture inutile sur le disque : si une page n'a pas été modifiée ($D=0$), sa copie sur le disque est toujours à jour et elle peut être simplement écrasée en mémoire.

Compression de la Mémoire

Introduite dans Windows 10 et d'autres systèmes modernes, la compression de la mémoire est une alternative au "swapping". Au lieu d'écrire les pages inactives sur le disque, le système les compresse et les conserve dans une zone dédiée de la RAM. L'accès à une page compressée déclenche un défaut de page, l'OS la décompresse et la restaure. Cette opération est beaucoup plus rapide qu'un accès disque.

15.5.Défis et Solutions d'Implémentation

La Taille des Tables de Pages

Avec un adressage 32 bits et des pages de 4 Ko, une table de pages à niveau unique nécessiterait plus d'un million d'entrées. Si chaque entrée fait 4 octets, la table elle-même occuperait 4 Mo par processus, ce qui est très inefficace.

- **Solution : Tables de Pages à Plusieurs Niveaux** Au lieu d'une seule grande table, on utilise une hiérarchie. Par exemple, une **table de répertoires de pages (Page Directory)** de premier niveau contient des pointeurs vers des **tables de pages (Page Table)** de second niveau. Celles-ci contiennent les pointeurs vers les trames physiques. Si une large plage d'adresses virtuelles n'est pas utilisée, il n'est pas nécessaire d'allouer les tables de pages de second niveau correspondantes, ce qui économise une quantité significative d'espace. Pour les adresses 64 bits, cette hiérarchie peut atteindre 4 niveaux (PML4, Page-Directory-Pointer Table, Page-Directory, Page Table).
- **Inconvénient** : Cette hiérarchie augmente le nombre d'accès à la mémoire physique nécessaires pour une seule traduction d'adresse (jusqu'à 5 pour le modèle à 4 niveaux).

Mise en Cache de la Traduction : le TLB

Pour contrer la latence introduite par les tables de pages à plusieurs niveaux, les CPU intègrent un cache spécialisé pour les traductions d'adresses : le **Translation Lookaside Buffer (TLB)**.

- Le TLB stocke les traductions récentes ($\#page \rightarrow \#frame$).
- Lors d'un accès mémoire, la MMU consulte d'abord le TLB.

- **Succès TLB (Hit)** : La traduction est trouvée et est quasi instantanée (0.5 à 1 cycle d'horloge).
- **Échec TLB (Miss)** : La MMU doit parcourir la hiérarchie des tables de pages en mémoire, ce qui est beaucoup plus lent (10 à 100 cycles), puis stocke la nouvelle traduction dans le TLB.
- Les CPU modernes ont souvent plusieurs niveaux de TLB (L1, L2). Lors d'un changement de contexte, le TLB doit être géré pour éviter qu'un processus n'utilise les traductions d'un autre. Les CPU modernes incluent un identifiant de processus (ID) dans les entrées du TLB pour éviter d'avoir à le vider à chaque changement.

Les Pages Géantes (Huge Pages)

Une autre façon de réduire les échecs du TLB est d'utiliser des pages plus grandes (par exemple, 2 Mo ou 1 Go). Une seule entrée de TLB peut alors couvrir une plus grande plage de mémoire, augmentant la probabilité d'un succès TLB. Cependant, l'allocation de grands blocs contigus de mémoire physique pour ces pages peut être difficile à réaliser.

15.6.Interaction avec les Caches du CPU

La question se pose de savoir si le cache principal du CPU doit utiliser les adresses virtuelles (VA) ou physiques (PA).

- **Caches à Adressage Virtuel (VIVT)** : Le CPU accède au cache avec la VA. Le TLB n'est consulté qu'en cas de cache-miss. C'est plus rapide, mais pose des problèmes de cohérence (l'OS doit vider manuellement les entrées de cache si un mappage de page change) et d'aliasing (deux VA différentes mappées sur la même PA sont vues comme des données distinctes par le cache).
- **Caches à Adressage Physique (PIPT)** : Le CPU traduit d'abord la VA en PA via le TLB, puis accède au cache avec la PA. C'est plus simple à gérer et évite l'aliasing, mais fait du TLB un goulot d'étranglement potentiel, car il doit être consulté à chaque accès mémoire.

15.7.Aspects de Sécurité

Modes Privilégié et Utilisateur

Pour protéger le système, les CPU fonctionnent dans au moins deux modes :

- **Mode Privilégié (ou Superviseur)** : Utilisé par le noyau de l'OS. Toutes les instructions et ressources sont accessibles.
- **Mode Utilisateur** : Utilisé par les applications. Les instructions critiques, comme la modification du registre pointant vers la table des pages, sont interdites.

Un processus utilisateur exécute une instruction spéciale pour appeler une fonction de l'OS. Le CPU passe alors en mode privilégié, exécute le code de l'OS, puis retourne en mode utilisateur.

Protection de la Table des Pages

Un processus utilisateur ne doit pas pouvoir modifier sa propre table des pages, car cela lui permettrait de contourner l'isolation. La solution la plus courante consiste à inclure les données et tables de pages du noyau de l'OS dans l'espace d'adressage de chaque processus, mais en les protégeant avec un indicateur **User/Supervisor (U/S)**. Les pages marquées $U=0$ ne sont accessibles qu'en mode privilégié. Toute tentative d'accès depuis le mode utilisateur déclenche un défaut de page, assurant la protection du système.

15.8. Annexe : Multitâche et Changement de Contexte

Sur un CPU à un seul cœur, le multitâche est simulé par le **partage de temps (time slicing)**. Le système d'exploitation alloue à chaque processus un court intervalle de temps (par exemple, 20 ms) pour s'exécuter. À la fin de cet intervalle, l'OS interrompt le processus, sauvegarde son état complet (registres, pointeur d'instruction, etc.) – une opération appelée **changement de contexte (context switch)** – puis charge l'état d'un autre processus et lui donne la main. Si ce changement est assez rapide, l'utilisateur a l'impression que les programmes s'exécutent simultanément.

Le changement de contexte est une opération coûteuse car il implique des accès mémoire pour sauvegarder/restaurer l'état et invalide le contenu des caches du CPU, qui contenaient les données du processus précédent. Il existe donc un compromis entre la réactivité pour l'utilisateur (intervalles courts) et la performance globale du système (intervalles plus longs pour minimiser les changements de contexte).